

UniAssist FYP-2 — Master Specification

Complete, unabridged: Rev 5 + Rev 6 + the SaaS pivot + Rev 7's critical review, in one document

UniAssist — Master Specification (Complete)

Single file containing the full, unabridged text of every planning document produced for UniAssist FYP-2, in one place, so nothing has to be cross-referenced across separate PDFs. This file is the complete reference; a separate well-structured PDF (UniAssist_FYP2_Spec_Rev7_Final.pdf regenerated from this same source) is the human-readable presentation of the same content.

How to read this file — precedence when parts disagree

The four parts below were written in this order: **Part A (Rev 5)** → **Part B (Rev 6)** → **Part C (SaaS Plan)** → **Part D (Rev 7)**. Each part is preserved in full and unedited from its source document — nothing is trimmed. Where a later part changes an earlier decision, the later part wins. The cross-reference table below is the fast way to find the current, governing answer without reading all four parts in full.

Decision area	Governed by	Current answer (one line)
Deployment / tenancy model	Part D §8 (supersedes Part A §2.2 and Part B)	Database-per-tenant, one shared MongoDB cluster + one shared ChromaDB server, not single-tenant.
Role hierarchy	Part D §7 (supersedes the draft matrix in Part C)	Super Admin → Administrator → Dept. Rep → Student → Guest.
Super Admin identity	Part D §8.3	Separate SuperAdminUser collection in a new PlatformDB, not a 4th User .role value.
Tenant request routing	Part D §5.5 (supersedes Part C §6's subdomain default)	JWT-embedded tenant claim, not subdomains.
ChromaDB tenancy mechanism	Part D §5.4 (corrects Part C §6)	One PersistentClient, one path, one collection per tenant.
CORS fix	Part B Finding 3, reaffirmed by Part D §5.6	Single CLIENT_URL allow-list — unchanged, because JWT-claim routing means one frontend origin.
Login lockout / OTP limiter / stray ChromaDB	Part B Findings 1, 2, 4	Still open — dead fields, unwired middleware, untracked stray store; fixes described in Part B.
Feature set (Scholarships/Jobs/Events/matching/etc.)	Part A, in full	Unchanged by the SaaS pivot — every module in Part A still applies, now running inside one tenant's database.
Build order	Part D §9 (supersedes Part A §12 and Part C §7 individually)	T0/T1 precede Part A's Phase 0; Part A's phases then proceed unchanged; T2/T3 run later.
Open items requiring verification	Part D §10 (merges Part A §16 with new items)	Single consolidated list — see Part D §10.
Scope cut for the SaaS layer	Part D §6	Must/Nice/Defer — see table there before adding SaaS-layer work.

PART A — Rev 5: Full Feature Specification (original, complete)

Preserved in full from UniAssist_FYP2_Spec_Rev5.pdf. Where the SaaS pivot changes a decision made here, Part D says so explicitly — this part is not edited to match.

UniAssist FYP-2 Spec ■■■ Rev 5 UniAssist — FYP-2 Feature Specification Revision 5 — final planning spec Project: UniAssist — Centralized AI Student Portal University: Muhammad Ali Jinnah University (MAJU) Repo: github.com/ibrahim123-sia/UniAssist Date: August 2026 Stack: React 19 + Redux Toolkit + Tailwind (client) · Node.js + Express + MongoDB (server) · FastAPI + ChromaDB + sentence-transformers (python) Revision history Rev 1 — original planning spec. Written without codebase access, so several assumptions in it were wrong. Rev 2 — produced after a full codebase audit. Corrected the wrong assumptions, added twenty engineering improvements across two rounds, and recorded a hard constraint change: Apify's paid scraping credits are exhausted, so all data acquisition must run on free infrastructure. Rev 3 — a review pass over Rev 2: CV privacy approach changed, build order reordered, engineering additions bucketed, three items dropped, RAG prompt and retrieval stage analysed. What changed: Change Why CV parsing moved from local Ollama to cloud API with PII stripping Ollama is not currently in use; the project runs on cloud APIs. The privacy commitment is kept through a different technique. Build order reordered — Scholarships before Jobs; Section 3 improvements moved much earlier Scholarships has no legacy code to reconcile. Section 3 is what actually improves the product for students. The twenty engineering additions split into three priority buckets Twenty individually-cheap items are not collectively cheap. Sequencing them prevents scope creep. Three items dropped, with reasons recorded See Section 15. Nothing is removed silently. Rework sequencing made explicit “Delete the old code” must not be read as “delete first, build later”. Golden-set evaluation moved ahead of retrieval changes You cannot tell whether a change improved retrieval without a baseline to measure against. Pydantic v1/v2 promoted from a note to a Phase 0 blocker It gates the extraction library choice. Rev 4 — closed the feature-level gaps found in a completeness review of Rev 3, and fixed two internal contradictions Rev 3 introduced.

Change Why Listing lifecycle added (§6.5) — expiry, delisting, and saved-item handling The pipeline only added and updated. Nothing expired, and a filled job would have stayed live forever. Delisting detection added to the pipeline (§7.5) Page-level hashing cannot detect a listing disappearing from a page that still exists. This needs per-source listing-set comparison — a real design change, not a note. Search specified (§6.6) — keyword only Filters and a table were specified; search never was. Notifications section added to the profile (§5.1) §10 said the student picks a digest frequency; no screen existed to pick it on. Staff role scope stated (§6.7) Three roles exist and the spec never said what two of them see in the new modules. Merge visibility and unmerge (§8.2, §9.5) Dedup was silent, yet the demo checklist treats it as a visible moment. Embedding similarity will also produce wrong merges that need undoing. Past events (§6.4) Undefined whether they disappeared or remained. Contradiction fixed: the full prompt deploys at Phase 3, not split across phases §8.7.1 said Branch D's widening surfaces at Phase 9, but §8.7.6 is one block that Phase 3 needs for confidence tiering. Contradiction fixed: golden set moved into Phase 1 §13.2 required every prompt change to be scored against the set, while placing the set after core work — but Phase 3 changes the prompt. The spec contradicted its own “measure before you change” principle. Rev 5 (this document) — adds the guest experience as a first-class concern rather than a reduced version of the student experience. Section 19 is new. Change Why Guest treated as a distinct user with a distinct goal (§19) Rev 4 treated a guest as “student minus features”. A guest is a prospective applicant deciding whether to apply — a different person asking different questions. {user_type} added to the RAG prompt (§8.7.6) Branch E currently tells users to file a portal ticket. A guest has no login, so that instruction is dead advice — a live bug. Internal / external scope flag on scholarships and events (§19.2) A guest should see MAJU's own scholarships and campus life, not DAAD and Chevening — external scholarships actively push a prospective applicant elsewhere. Guest landing page reframed as an admissions funnel (§19.3) Follows from the above. The page has a purpose now, not just reduced permissions. Guest chat history carried into registration (§19.5) A guest who talks to the bot for twenty minutes and then registers currently loses all of it. Guest chat rate limiting (§19.6) Unauthenticated LLM access is the project's largest free-tier exposure and Rev 4 did not cover it. Guest failed-question log kept separate (§19.4) Guest gaps are admissions-content gaps — a different and more commercially useful signal than student gaps. MAJU's own scholarships become a structured source (§19.2) Previously the university site was scraped only as unstructured chatbot chunks. Internal scholarships now need structured records too. Change appetite: existing pages, schemas and backend code are all open to change where the change makes the system better. Rev 5 touches the guest landing page, the RAG prompt, the scholarship and event schemas, and the scraping scope — all deliberately. No demo deadline is pending. This removes the pressure that would otherwise force patching over rebuilding, and is what makes the clean-rebuild policy in Section 2.3 affordable.

Rev5 §1. At a glance

New modules

Feature What it is Ref Scholarships module New backend pipeline targeting MS-abroad portals. CGPA scale conversion, deadline tracking, last-verified badges. No legacy code — built first. §6.2 Jobs module Own scraper (careers pages, ATS boards, remote-job APIs) feeding a matched feed + browse table + detail page. Replaces the Apify- dependent client-side implementation entirely. §6.3 Events module MAJU website scraping (extending scraper.py) + manual admin entry, personalized by department and interests. §6.4 Unified student profile One profile in four sections, replacing the current name/avatar/password-only settings page. §5 CV upload & parsing PII stripped → cloud extraction → editable pre- filled form → student confirms. Never silent- saved. §5.2 Matching engine Cascade: hard rules → embeddings → cross- encoder re-rank → LLM explanation → RAG. §8 Source management Paste-a-URL scraping, one-time or recurring, review queue, health monitoring, SSRF-safe validation. §9 In-house scraper Fetch → hash-check → LLM+Pydantic extraction → validate → dedupe → pending. Shared across all three modules. §7 Notifications & digests Digest emails, deadline reminders, saved-item change alerts, via a transactional provider. Frequency set in the profile's Notifications section. §10 Listing lifecycle Expired and delisted states; saved items keep history with a badge. Nothing hard-deleted. §6.5 Delisting detection Per-source listing-set comparison — catches a listing vanishing from a page that still exists. §7.4 Keyword search Text search over title, organization and description. Semantic querying stays with the chatbot. §6.6 Merge visibility & unmerge Merges shown to students as “found on N sources” and to admins as a reviewable, reversible action. §8, §9.5 Guest admissions funnel Guest treated as a prospective applicant: internal scholarships, campus events, context- aware chatbot chips. §19 Guest-aware prompt {user_type} branching — fixes a live bug where guests are told to file portal tickets they cannot access. §19.4 Guest rate limiting & caching The one unauthenticated LLM endpoint in the system, and its largest free-tier exposure. §19.6 Improvements to existing modules

Feature What it is Ref Chatbot → query handoff Three-tier confidence with retrieval-based department suggestion. §4.2 Failed-question log Low-confidence questions logged, grouped, surfaced to admin with frequency. §4.3 SLA escalation Configurable threshold, background breach check, escalation flag and filters. §4.1 Issue feedback Thumbs up/down + rating on resolved issues, surfaced in dashboards. §4.4 Proactive notifications Deadlines, saved-item changes, stale queries, department events — pushed, not waited for. §4.5 Config-driven branding University name/domain/colors/logo from config, not hardcoded. §11 Engineering additions — bucketed by when Bucket Items Ref Now — build alongside core Shared student components · helmet.js · CORS lock-down · auth rate limiting · login lockout · startup config validation · ChromaDB backup §13.1 Phase 1 — gates everything downstream Golden-set evaluation (moved out of “next” in Rev 4) §13.2 Next — right after core Targeted unit tests · health check page · near- miss bucket · department middleware §13.2 Later — post-FYP-2 Cross-encoder re-ranking · grounding check · RAG caching · matching audit trail · data export/delete §13.3 Dropped Admin-panel LLM keys · LiteLLM refactor · engagement feedback loop §15

Rev5 §2. What this document is, and the rules it follows

Rev5 §2.1 Purpose

This is the working spec for FYP-2. It covers three things:

1. New modules deferred from FYP-1 — Jobs, Scholarships, Events, Profile, Content Management
2. Improvements to modules already built — mainly the chatbot and the query system
3. Engineering hardening identified during the codebase audit

Some items here are locked decisions. Some are still unverified and must be checked before coding — those are in Section 16, and nothing should be architected around them until they are confirmed.

Rev5 §2.2 Deployment model

Single-tenant. One university per instance. The product gets rebranded and redeployed for each new university. This is not a multi-tenant SaaS. No tenant_id work is needed anywhere, and that is a deliberate simplification — multi-tenancy would mean touching every model and every query, for a capability the business model does not require.

Rev5 §2.3 Rework policy

Where a new approach replaces an existing feature, the old implementation is removed outright. No compatibility shims, no feature flags to switch between old and new, no dead code left behind. But the sequence matters, and it is not “delete first”:

1. Build the new implementation
2. Verify it works end to end

3. Cut over

4. Then delete the old code

Deleting Job.jsx before its replacement runs would leave the feature broken for however many weeks the rebuild takes. That is avoidable with no cost. Before deleting, record the commit hash (or keep a branch). The codebase stays clean — the old code does not sit in the working tree — but it remains reachable in git history. Two concrete reasons: Job.jsx contains roughly 1300 lines of regex parsing. Some of that field-extraction logic works (salary parsing, location normalization) and is worth consulting while writing the replacement. The report benefits from a before/after comparison, which needs the “before” to exist somewhere. Do one module at a time. Finish Scholarships completely, then start Jobs. Three half-rebuilt modules at once means one blocker breaks all three.

Rev5 §2.4 Design principles

These are the reasons behind the specific decisions later in the document. When a detail is ambiguous, resolve it against these. Closing loops beats showing information. The core weakness of the FYP-1 feature set is that everything displays something and the student still has to go do the work. A chatbot answers, then the student goes elsewhere. A listing appears, then the student applies elsewhere. Every design choice in this spec should be checked against: does this close a loop, or does it just add another thing to look at? Cheap checks before expensive ones. Every pipeline in this spec is a cascade — a plain database query filters most candidates before any embedding runs, and embeddings narrow the field before any LLM call happens. This is not premature optimization; it is what keeps the whole system inside free API tiers. Don't recompute what hasn't changed. Hash-based change detection for scraping (§7.2), content-hash cache keys for LLM explanations (§8.4), write-time embeddings rather than read-time (§8.2). Same principle, three places. Config over code. Sources live in the database. Branding lives in environment config. Neither should require a deploy to change. Measure before you change. Any modification to the RAG pipeline needs a baseline to compare against. This is why the golden set (§13.2) comes before the cross-encoder (§13.3), not after.

Rev5 §3. Where the project stands today

Already built (FYP-1) Student side — Guest landing page. Register/login with MAJU email, OTP verification, password recovery. AI chatbot: RAG over MAJU website data, voice input via local Whisper, Roman Urdu support, profanity filter with admin flagging. Issue tracker: attachments, department routing, threaded replies, status tracking, 20-second polling, in-app bell plus email. Staff side (department-scoped) — Dashboard with open/in-progress/resolved cards, 7-day trend, average first-response time. Department inbox with live status filters. Issue detail with full thread. Status management. Threaded replies with notifications. Admin side — Dashboard. Users with search, filters, block/unblock. Staff with auto-generated credentials. Departments CRUD. Query analytics. Vector DB CRUD with document upload and auto-chunking. Logs and activity: audit trail with redacted payloads, login events, chat browser. Audit findings that changed the plan These came out of the codebase audit and corrected assumptions in Rev 1: Jobs and Events are live features, not stubs. They run entirely client-side against external Apify endpoints, parsing raw text with regex. No backend persistence, no admin moderation. Only Scholarships is a genuine placeholder. Apify credits are exhausted. Jobs and Events are therefore non-functional right now. There is no paid fallback and no interim option — the in-house pipeline is mandatory, not preferred. rag.py already computes a confidence score (combined semantic + keyword) but discards it — the variable is prefixed _scores and never surfaced. AnswerResponse has no confidence field. This makes §4.2 much cheaper than it looked. User.js has zero academic fields. No CGPA, skills, IELTS, or degree. The profile is greenfield schema work. No cron or scheduler infrastructure exists anywhere — Node or Python. SLA escalation, recurring scrapes, and digest emails all

need this built. No hashing or change detection exists anywhere. scraper.py is MAJU/WordPress-specific. Its CSS-selector logic does not generalize to arbitrary sites. Only its HTTP-fetch plumbing and chunk_elements() helper carry over. requirements.txt pins pydantic==1.10.13 . This gates the extraction library choice — see §16. auditAdminWrites middleware auto-logs any non-GET admin route mounted under adminRoutes.js . New routes get audit logging free — but its targetId extraction only recognizes user/department/issue response keys and needs extending. Zero automated tests exist anywhere in the project. No HTTP security headers are set. express-rate-limit is installed but its application to auth endpoints is unverified. Pending Rebuilding Jobs, Scholarships and Events on an owned backend pipeline. Unified profile. Content and source management. Matching engine. The improvements in Section 4. The engineering additions in Section 13.

Rev5 §4. Improvements to existing modules

These come first in the build order — ahead of most of the new modules — because they are what actually make the product better for the student who uses it. The engineering additions in Section 13 are polish; these are substance.

Rev5 §4.1 SLA escalation on issues

Problem: an issue can sit unanswered indefinitely and nobody notices. “Resolution rate” looks fine because unanswered issues simply stay open. Build: - Configurable SLA threshold, default 48 hours from creation to first staff reply - Background job checks for breached issues - On breach: notify department head and admin; set escalated flag and escalatedAt timestamp on the issue - Escalated issues get a dedicated filter in both staff inbox and admin panel - Admin can configure the threshold per department (a fee query and a transcript request have different reasonable turnarounds) Example: a student files a query about a fee challan on Monday morning. Nobody in the Finance department opens it. Wednesday morning the background job flags it, the department head gets a notification, and it appears in the admin’s escalated filter with “48h breach, no first response.” Why this matters more than it looks: this is the single change that turns a ticket system into an accountability system. It also produces the one number university administration actually responds to — “average first response dropped from three days to six hours” is a procurement argument in a way that “we have an AI chatbot” is not. Codebase note: Issue.js has no SLA fields today, and no cron infrastructure exists. This is net-new schema plus net-new scheduler — not a small patch. The scheduler built here is reused by recurring scrapes (§9) and digest emails (§10), so the cost is shared.

Rev5 §4.2 Chatbot → query handoff

Problem: the chatbot and the issue tracker are two disconnected systems. A student asks the bot, gets nothing useful, then separately navigates to the issue form and retypes everything. Build — three confidence tiers, not two: Tier Condition Behaviour Confident Good retrieval score, relevant chunks found Answer normally Ambiguous Some relevant chunks, borderline score Ask one clarifying question before answering Low confidence No relevant chunks, or score well below threshold Offer to file a query to the relevant department Why three and not two: a binary split sends every uncertain question straight to an escalation offer, including ones the bot could have answered with a single follow-up. If a student asks “when is the deadline?” and the knowledge base has three different deadlines, the right response is “which deadline — fee submission, course registration, or thesis?” — not a query-handoff form. On handoff: one click creates the issue, with the chat transcript attached automatically. The student does not retype anything. Department suggestion via retrieval, not keywords: store labeled example question→department pairs. For a new question, retrieve the most similar examples and let those guide the suggestion. The student can override. Confusion tracking: when a student changes the suggested department, log the suggested-vs-corrected pair. Periodically review the

resulting confusion matrix to find department pairs that get mixed up — for example, if Fee Office and Finance Office are confused fifteen times, that specific pair needs disambiguation. This is targeted rather than guessing where the problems are. Codebase note: rag.py already computes the score needed for tiering and throws it away. Surfacing it through AnswerResponse and into the Node message controller is the main wiring work.

Rev5 §4.3 Failed-question log

Problem: nothing makes the chatbot better over time. It answers what it can and silently fails at the rest. Build: - Log every low-confidence or unanswered question - Admin view listing them, grouped by topic similarity using the existing embeddings - Show frequency — “asked 14 times this month” - Direct jump from this view into the existing Vector DB CRUD screen to add the missing content Example: twenty-three students ask some variation of “how do I apply for a semester freeze?” over a month. None get an answer because that policy was never added to the knowledge base. The admin sees one grouped row with a count of 23, uploads the policy document, and the next student gets an answer. Why this pairs with §4.2: together they form a self-improving loop. Questions the bot fails become the queue for improving the bot. This is genuinely interesting technically, it is easy to demonstrate, and it gets better with use rather than degrading.

Rev5 §4.4 Issue feedback and satisfaction

Problem: “Resolved” only means a staff member clicked a button. It does not mean the student’s problem was solved. A department that closes tickets without helping looks identical in the metrics to one that helps. Build: - On transition to Resolved, prompt the student: was this helpful? Thumbs up/down, optional 1–5 rating, optional comment - Store on the issue - Surface satisfaction rate alongside resolution rate in staff dashboard and admin query analytics - Admin sees per-department satisfaction, not just per-department volume Example: Department A resolves 95% of issues with 40% satisfaction. Department B resolves 70% with 90% satisfaction. Without this feature, Department A looks better. With it, the actual picture is visible.

Rev5 §4.5 Proactive notifications

Instead of waiting for the student to come and look, the system reaches out: Fee, admission, and exam deadlines drawn from the chatbot knowledge base A saved scholarship or job whose details changed A saved item with an approaching deadline The student's own query sitting unanswered Events in the student's department this week All of these respect the digest settings in Section 10 — the point is to be useful, not to generate volume.

Rev5 §5. Unified student profile

Rev5 §5.1 Structure

One profile, not three. Earlier planning had separate “job profile” and “scholarship profile” — dropped. Section Fields Core Name, degree program, CGPA, current semester, expected graduation date, department Career Skills, work mode preference (remote/onsite/hybrid), preferred city, CV, experience level Study abroad IELTS/TOEFL score, target countries, funding preference, intended field of study Events Interests, society memberships Notifications Digest frequency (daily/weekly/off), which modules to include, deadline- reminder lead time, unsubscribe-all Core feeds everything. The other sections feed their respective modules.

Why one profile: a student who enters an IELTS score for scholarships also benefits in job matching, since international remote roles frequently list English requirements. Split profiles lose that connection and force the student to enter CGPA three separate times. Why Notifications lives in the profile rather than a separate settings page: §10 commits the student to choosing a digest frequency, but Rev 3 gave them nowhere to choose it. Putting it here keeps everything the student controls about themselves in one place, and it sits naturally next to the fields that drive the matching those digests are about. It is also the only section that is not used by matching, so it should be visually separated from the other four. Codebase note: User.js has no academic fields today. A nested profile object is the least invasive approach — existing auth and login code stays untouched.

Rev5 §5.2 Two ways to fill it

Path A — CV upload Upload (PDF/DOCX) → Extract text → Strip PII → Cloud LLM parses into structured fields → Show pre-filled EDITABLE form → Student reviews and corrects → Confirm → Save Path B — manual. Plain form, same fields. CV upload is the default; manual is a visible secondary link. A student who filled the form manually can upload a CV later to fill gaps. Never save silently. Two reasons:

1. Extraction will make mistakes. “CGPA 3.4/4.0” gets read as 4.0. Date ranges get inverted. A GPA on a different scale gets copied as-
is. If the student never confirmed it, wrong matching results are blamed on the system — correctly.
2. The confirmation step is itself a good demo moment — “the AI extracted this from the CV, the student just verifies it” is more convincing than data appearing from nowhere.

Rev5 §5.3 CV privacy — PII stripping

Changed in Rev 3. Rev 2 specified local Ollama parsing. The project now runs on cloud APIs and Ollama is not in active use, so the privacy commitment is kept through a different technique. Strip before sending to any external API: Remove Keep Phone numbers Degree, university, graduation year Email addresses CGPA Physical address Skills CNIC / ID numbers Projects Date of birth Work experience Photo Certifications, languages Name — The right-hand column is everything matching actually uses. The left-hand column is not needed for any part of the system. Delete the raw CV file after parsing. Structured fields go to MongoDB; the file does not persist. Without this the PII stripping is pointless — the unredacted document would still be sitting on the server. Tell the student. One line on the upload screen: “Your personal details (phone, email, address) are removed before processing.” Small thing, builds trust, and it is a real design decision worth writing up in the report. Why this is as defensible as local-only processing: free API tiers commonly reserve the right to use inputs for model improvement. The concern is real. Stripping PII addresses it directly, and unlike local-model processing it does not depend on which model happens to be running or how much RAM is available.

Rev5 §5.4 Access — soft banner, not a hard gate

Students browse jobs, scholarships and events without a completed profile. They see the unfiltered list plus a banner. Do not gate access. Two reasons:

1. Asking for work before demonstrating value is the standard way to lose users. A student who lands on an empty form and does not know what is behind it leaves.

2. The unfiltered-versus-matched comparison is the best available demonstration of the matching engine. Show the raw list, fill in the profile, show the same data ranked with eligibility reasoning. That contrast cannot be shown if the unfiltered view does not exist. Banner should be specific: Weak: “Complete your profile” Better: “42 scholarships available. Add your CGPA and IELTS score and we’ll show which ones you’re eligible for.” Progressive: “Profile 60% complete — add target countries for better matches” On the profile page, label each section with what it unlocks: “IELTS score → used for scholarship eligibility.” “Skills → used for job matching.” The student should know what filling a field buys them. One place a gate is correct: email alerts. No profile means no matching, so no alerts. Say that plainly.

Rev5 §6. The three modules

All three share the same shape. Only the schema differs.

Rev5 §6.1 Common UI pattern

Matched feed — the primary landing view. “7 scholarships match your profile. You’re fully eligible for 3.” Browse table — full list, all filters, sortable. Secondary. Detail page — complete information plus redirect link to the original source Saved / bookmarked items Application tracker (jobs and scholarships) Deadline calendar view The matched feed being primary is the whole point. A plain table is a directory — Rozee is a directory, and it already exists. The matched feed is what makes this UniAssist and not UniList. If the student has to go find things themselves, the system has not helped. Application tracker is self-marked, not automatic. There is no way to detect whether a student actually applied — the system only holds a redirect link. The flow is: student clicks Apply, gets redirected, returns, and a small prompt asks “did you apply?” Yes sets the status and date. Beyond that the student updates it themselves — Interviewing when a call comes, Selected or Rejected when the result arrives. This is a personal notebook, not an integration. Its value is that a final-year student applying to fifteen places forgets where they applied and what happened. It also makes the reminders in §4.5 possible: “you applied here three weeks ago — any update?”

Rev5 §6.2 Scholarships — build this first

Target: MS-abroad opportunities for final-year students. Why first: this module has no working precedent in the codebase — only a static placeholder. That means the pipeline gets built once, clean, with no legacy implementation to reconcile against. Jobs and Events then reuse a pipeline that has already been proven. Sources: centralized portals, not individual universities. Individual universities have no APIs, no consistent page structure, and will not partner with a student project — outreach emails go unanswered and weeks get lost waiting. Centralized portals each cover thousands of programs and have far more stable page structures. Displayed detail: program name, university, country, degree level, funding type, eligibility criteria, required documents, language requirements, deadline, official link, last-verified date. CGPA scale conversion. Pakistani 4.0 scale to German 1–5 and to percentage. This is a genuine value-add — it is a real problem students cannot solve themselves and get wrong constantly. It is also pure deterministic logic, which makes it one of the four things worth unit- testing (§13.2). “Last verified” date on every entry, with a warning badge past 30 days. This is what makes the change-detection pipeline visible to the student instead of being an invisible backend optimization.

Rev5 §6.3 Jobs

Scope: all kinds — remote, onsite, hybrid, local and international. Bucket Method Notes Local (Karachi/Lahore, onsite + hybrid) Scraping company careers pages, possibly Rozee Primary local relevance Remote-for-Pakistan ATS boards of staffing companies hiring in Pakistan Free, clean data Global remote Remote job board APIs Free, high volume Important finding — do not assume APIs cover Pakistani jobs. Greenhouse and Lever are primarily US and European ATS platforms. Most ordinary Pakistani companies post on their own careers pages, on Rozee, or on LinkedIn instead. Some ATS boards do carry Pakistan roles, but they tend to belong to remote-staffing companies hiring senior developers rather than local fresh-grad employers. Scraping is the primary route for local roles. Verify actual coverage (§16) rather than planning around an assumption. LinkedIn: do not scrape. Aggressive anti-bot, explicit ToS prohibition, real legal precedent. This applies with extra force now that Apify is being retired — building an in-house LinkedIn scraper would recreate exactly the risk the original Apify approach was avoiding. Schema fields beyond the obvious: Field Why experience_level Extract from JD text, not just the title. “Engineer” in the title with “5+ years” in the body is a senior role. work_mode remote / onsite / hybrid location_restriction Many remote listings are region-locked (“US-based only”, specific timezones) and this is buried in description prose, never in a structured field. is_fresh_grad_friendly Boost internship, junior, associate, “0–2 years”, “fresh graduate” Without these fields the feature fails. A feed of 500 jobs where three are applicable by a final-year student is worse than no feed. This filtering is the differentiator versus Rozee — the data is the same data, the judgement is not. Skill gap analysis. The CV is already parsed, so comparing it against a JD is nearly free: “this role wants Docker and AWS, which aren’t on your CV.” Rebuild note: the current Job.jsx — client-side,

Apify-dependent, roughly 1300 lines of regex parsing — is retired in full. Follow the sequencing in §2.3: build, verify, cut over, then delete.

Rev5 §6.4 Events

Sources: two MAJU websites (partly handled by the existing scraper.py) plus manual admin entry. Reality check before building anything. Pakistani universities often post events as poster images rather than structured text, and much of it goes to Instagram or Facebook rather than the website. Check first: How many events actually appear on the MAJU sites in a semester? Text or images? If the volume is low, manual entry with a good admin UI is entirely acceptable and no automation is justified. If events are images, plain scraping returns nothing and a vision model or OCR step is needed — a different problem from scraping. Also worth doing: ask student affairs, through the supervisor, whether they will provide event data directly. This is the one area where a human conversation beats any scraper — MAJU is your own university and they are reachable, unlike DAAD. It also reads well in the report. Personalization: filter and rank by department, semester, and interests. Past events stay, in a separate tab. They are excluded from the matched feed and from alerts, but remain browsable. Deleting them loses the record of what the university actually ran, which is useful both to students looking for recurring annual events and to the report. Rebuild note: the current Events.jsx (Apify LinkedIn-caption scraping) is retired in full.

Rev5 §6.5 Listing lifecycle

Added in Rev 4. Rev 3's pipeline only added and updated listings. Nothing ever expired or was removed, which means a scholarship whose deadline passed in January would still appear in March, and a job filled last month would still be listed. Three distinct cases, each handled differently. Nothing is ever hard-deleted — history is needed for the application tracker, for the report, and for debugging what the scraper did. Case Trigger Handling Expired Deadline date has passed Status → expired . Removed from matched feed and from default browse view. Still reachable, and still visible to any student who saved it, with an “Expired” badge. Delisted The listing disappears from its source page — job filled, programme withdrawn Status → delisted . Same visibility rules as expired. Detection requires the pipeline change in §7.5. Saved item expires or is delisted Either of the above, on an item a student saved One notification (“this closed”), the item stays in their saved list with a badge, and any tracker entry keeps its history. Why the distinction between expired and delisted matters: expired is predictable and can be computed from a date field with no scraping at all — a daily job flips the status. Delisted requires actively noticing an absence, which is a harder problem and the reason §7.5 exists. Conflating them would hide the fact that only one of the two is nearly free. A delisted item is not necessarily gone. Sources sometimes reorganize pages, and a listing can vanish and reappear. So delisted is a status, not a deletion, and a subsequent scrape that finds the listing again simply restores it to approved .

Rev5 §6.6 Search

Keyword search only — over title, organization or university, and description. Standard text search, no embeddings. Why not semantic search here: the chatbot (§8 Layer 4) already provides semantic querying over the same content. Offering semantic search in two places duplicates the work and, worse, leaves the student unsure which one to use for what. The division should be legible: the table is for looking things up, the chatbot is for asking questions. A student who knows the word “Chevening” wants keyword search; a student who wants “fully funded, no IELTS, Germany” wants the chatbot. Search sits alongside filters, not instead of them. Filters handle the structured fields (country, funding type, work mode); search handles names and free text.

Rev5 §6.7 Staff role in the new modules

Three roles exist and Rev 3 never said what two of them see here. Jobs and Scholarships: nothing. These are student-facing features. Department staff have no role in them, and adding one would create moderation work nobody asked for. Events: one narrow capability worth having. Department staff can submit an event for their own department, which lands in the admin review queue like any scraped entry. Why this exception is worth it: department representatives know about their own department's events; the admin generally does not. This is exactly the manual-entry path from §9.1, opened to the people who actually hold the information — and it routes through the same review queue, so it adds no new moderation surface. It also fits the existing department-scoping pattern in the codebase. This is optional. If time is short, admin-only entry works and staff submission can wait.

Rev5 §7. Data pipeline

Constraint: Apify credits are exhausted and the project runs entirely on free infrastructure. The in-house pipeline is mandatory for all three modules — there is no interim fallback.

Rev5 §7.1 Architecture

One pipeline, three schemas.

Source (from DB, not hardcoded) ↓ Fetch (Crawl4AI) ↓ Hash check — unchanged? stop here ↓ Extract (LLM + Pydantic schema) ↓ Validate (schema check) ↓ Deduplicate (embeddings) ↓ Store as status = pending ↓ Admin review → approved → live

Rev5 §7.2 Change detection

Store a content hash per source page. On each scheduled run: fetch, hash, compare. Same hash → skip entirely. No LLM call. Different → extract, update, flag for review. This is not an optimization, it is the cost model. Without it, 500 pages hit the model every week. With it, only the 10–20 that actually changed do. That difference is what keeps the system inside free tiers — and with no paid Apify fallback to absorb overflow, there is no margin for getting this wrong. Codebase note: no hashing or change detection exists anywhere. Build from scratch. This is one of the four functions worth unit-testing (§13.2) — a silently broken hash comparison either burns the entire API budget or stops updating data, and both failure modes are invisible until they hurt.

Rev5 §7.3 Extraction approach

Do not write CSS-selector scrapers. They break the moment a site adjusts its layout, and maintaining twenty of them is not feasible. Instead: fetch the page, clean the text, hand the LLM a Pydantic schema, get JSON back. This survives layout changes. Crawl4AI must be an actual dependency, not a diagram label. It appears in the Rev 2 architecture diagram but is not in requirements.txt — the scraper today is plain requests + BeautifulSoup, which cannot render JS-heavy pages. Crawl4AI is open-source and self-hosted (no per-request credits to exhaust, unlike the Apify dependency being retired), renders JS via Playwright, and outputs clean LLM-friendly markdown. Use Instructor for the extraction call. It wraps an LLM call with Pydantic validation and retry built in, replacing hand-written “call, parse JSON, validate, retry” glue. Blocked on the Pydantic version decision — see §16. Avoid hosted scraping-as-a-service. Firecrawl and similar solve the same problem as Crawl4AI but as a credit-metered third-party API — precisely the risk profile that just materialized with Apify. Self-hosted only. Model routing: Task Model Reason Public pages (scholarships, jobs, events) Free hosted tier, with failover Public data, no privacy concern, and long HTML needs a capable model CV parsing Cloud API, PII stripped first (§5.3) Sensitive content handled by removing the sensitive parts Codebase note: rag.py 's existing multi-key failover client code is directly reusable for extraction calls — no new provider integration needed, only a new prompt and Pydantic schema per entity type. scraper.py 's CSS-selector logic is MAJU/WordPress-specific and does not generalize; only its HTTP-fetch plumbing and chunk_elements() helper carry over. Validation: run every LLM output through Pydantic. Missing or malformed required fields keep the entry in pending and flag it. Never push unvalidated data live — a wrong deadline shown to a student is worse than no deadline at all, because they will act on it.

Rev5 §7.4 Delisting detection

Added in Rev 4. This is a pipeline design change, not a note.

§7.2's page-level hashing answers one question: did this page change? It cannot answer the question delisting requires: did this particular listing disappear from a page that still exists? A careers page that drops one filled role and keeps twelve others has a different hash, so the pipeline knows something changed — but with only a hash it has no way to work out that a specific listing is gone. The change: track, per source, the set of listing identities seen on the most recent successful run. Run scrape on source S → extract listings → build identity set for this run → compare against the identity set stored from the previous run present now, absent before → new listing → pending present both times → update if changed absent now, present before → mark delisted (§6.5) → store this run's identity set on the Source record Listing identity should be a stable field or hash — the source's own listing URL where one exists, otherwise a hash of title plus organization. Do not use a database _id, which changes nothing about whether the same job is being described. Two guards against false delisting:

1. Only act on a fully successful run. A partial fetch, a timeout, or a page that rendered empty must not be read as “every listing disappeared.” If the run failed or returned suspiciously few listings, skip the comparison entirely and let the failure counter in §9.4 handle it.
2. Require two consecutive absences before marking delisted. This absorbs transient rendering failures and pagination quirks, at the

cost of one extra scrape cycle of latency — an acceptable trade for not wrongly hiding live listings. Cost note: this comparison is free. It runs on already-extracted output, needs no additional LLM call, and adds one array to the Source record. It is only being called out because the hash-based design in §7.2 does not cover it and would silently never expire anything.

Rev5 §7.5 Politeness and safety

3–5 second delay between requests to the same host Descriptive User-Agent including a contact email Respect robots.txt Never hammer small company servers That last point is practical, not just ethical: a small Pakistani company's site will block the IP, and you will discover it at the worst possible time.

Rev5 §8. Matching engine

This is the core of the product, not a feature on top of it. The scraper feeds it; the UI displays its output. Build it as a cascade — cheap and deterministic first, expensive and probabilistic last. Layer 1 — Hard rules (plain code, no LLM) CGPA $\geq$ required IELTS/TOEFL $\geq$ required Deadline has not passed Degree level matches Work mode and location compatible Fast, free, deterministic. Filters out most of the pool. Output is three-state, not boolean: State Meaning Hard-excluded Deadline passed, or degree level does not match at all. Never becomes eligible. Eligible Every requirement met. Near-miss A numeric requirement missed by a small configurable margin — e.g. ≤ 1.0 IELTS or ≤ 0.3 CGPA.

For every non-excluded candidate, Layer 1 also records which fields fell short and by how much. This structured gap data feeds the near-miss bucket (§13.2) and the explanation step directly — neither has to recompute it. Worth unit-testing (§13.2): pure deterministic logic, no mocking required, and a silent break here corrupts everything downstream. Layer 2 — Embeddings (ChromaDB, already in the stack) Semantic field matching: “Computer Science” $\leftrightarrow$ “Data Science & AI”. Keyword matching fails here; embedding matching works. Skill similarity Duplicate detection: the same job appears on a careers page and a job board with slightly different titles. Exact matching misses it; embedding similarity catches it. Precompute embeddings at write time, not read time. A listing's embedding is computed once when it is approved. A student's profile embedding is computed once when their profile is saved. A matched-feed request runs a similarity query against embeddings that already exist — it never computes them live. Feed load time then stays flat as the listing pool grows. Deduplicate at ingestion, not at match time. When a new scraped listing is about to be stored, check its embedding against existing approved listings of the same type. Above threshold, merge into the existing record's source list rather than creating a duplicate. One check per new listing, rather than a cost paid on every student query. Merges must be visible and reversible (added in Rev 4). Rev 3 made merging a silent ingestion-time operation, which conflicts with the demo checklist (§17) treating it as something to show. More importantly, embedding similarity will merge things it should not — two genuinely different roles at the same company with similar titles is the obvious case. Student-facing: the detail page shows “found on 3 sources” with links to each. This is useful information, not just a demo artifact — a student can check the original posting they trust. Admin-facing: merge events appear as a filter in the review queue, and an unmerge action splits a record back into its constituents. Without unmerge, a bad merge is permanent and the only fix is deleting and re-scraping. Log the similarity score on the merge so the threshold can be tuned from real cases rather than guessed.

Codebase note: ChromaDB's client is collection-name-agnostic, so a second collection alongside the existing chatbot knowledge base is straightforward. The chatbot's hybrid keyword-search layer is corpus-specific and should not be reused here — a simpler pure-semantic query fits matching better. Layer 2.5 — Cross-encoder re-ranking (deferred — see §13.3) A local cross-encoder scores query-candidate pairs more precisely than embedding similarity alone, at zero API cost. Run it only over the bounded candidate set Layers 1 and 2 have already narrowed to (roughly top 30–50) — a cross-encoder scores each pair individually and is far too slow to run across the whole pool. Deferred, deliberately. Deferring this is not a judgement on its value — it is sequencing. Without the golden-set baseline (§13.2) there is no way to tell whether adding it improved retrieval or quietly broke something. Build the measurement first. Layer 3 — LLM explanation (top ~10 only) Do not output a bare score. Output a reason: “You're eligible — this needs a 3.2 CGPA and yours is 3.4. But IELTS 6.5 is required and yours is 6.0. The deadline is 15 January, which gives you four months to retake it.” This is the difference between the product and a filtered list. Rozee gives a list. This gives a verdict and a next action. Cache explanations by content hash — the same principle as §7.2. Without caching, every matched-feed view fires up to ten fresh LLM calls; a student checking daily burns the free-tier budget for no benefit when nothing has changed. Cache key: hash(profile's matching-relevant fields) + hash(listing content) . There is no explicit invalidation logic to write. Changing either input changes the hash, which is a cache miss, which triggers a fresh explanation. The cache correctness follows from the key design rather than from remembering to invalidate. Layer 4 — RAG over the new content Reuse the existing RAG architecture, pointed at the scraped data: “Show me fully funded scholarships in Germany that don't require IELTS”

Do not reuse the prose-chunking strategy. The existing chunk_elements() is heading-anchored and paragraph-packed, built for MAJU's policy and FAQ pages. Job and scholarship listings are structured records, not prose. Chunking one listing by heading risks splitting its fields — deadline in one chunk, funding in another — so a query like “fully funded scholarships in Germany” could retrieve an incomplete fragment and answer from it. For this layer: one listing = one embedding. Embed the full structured record as a single document. Where the current RAG pipeline stands against standard practice. The architecture is standard —

hybrid retrieval (semantic + keyword), a vector store, sentence-transformer embeddings, chunking via a text splitter, multi-provider LLM with failover. What is missing is the quality-control layer that a mature pipeline has: Missing Status Confidence surfacing Score is computed in rag.py and discarded. Required by §4.2 anyway. Re-ranking after retrieval §13.3 — gated on the golden set Evaluation set §13.2 — the prerequisite for the other two Grounding / hallucination check §13.3 — gated on the golden set None of these are architectural problems. They are additions to a sound pipeline, and all four are already in this spec.

Rev5 §8.5 When the cascade runs

A hybrid timing model, not a single choice between on-request and nightly batch: Layer Timing Layer 1 Always fresh, on request. It is a plain Mongo query — no reason to cache or batch. Layer 2 Embeddings precomputed at write time; the similarity query runs on request. Layer 3 Lazily computed on first view after a relevant change, then cached until profile or listing changes. Why not a nightly batch for everything: a full nightly recompute for every student spends LLM calls on students who never open the feed, and still leaves results up to 24 hours stale for students who do. The hybrid model keeps the cheap layers always fresh and only pays the expensive cost when a real view needs it.

Rev5 §8.6 Explanation audit trail (deferred — see §13.3)

Log which profile fields were compared against which listing requirements to produce each explanation. Makes wrong matches debuggable and extends the project's existing auditability philosophy from admin actions to matching decisions. Cheap because Layer 1 already records the gap data. Deferred only because it is a debugging aid rather than something a student sees.

Rev5 §8.7 The RAG prompt

The prompt is part of the pipeline, not decoration around it. The current create_prompt() in rag.py is already mature — six-branch classification, pronoun resolution for elliptical follow-ups, explicit no-fabrication rules, consistency-with-history handling, and Roman Urdu phrasing guidance. Most standard RAG prompt advice is already implemented there. Five specific problems need fixing, two of which are direct conflicts with this spec. A sixth was found in Rev 5 and is a live bug rather than a future conflict: the prompt has no notion of who is asking, so Branch E tells guests to file portal tickets they have no account for. See §19.4.

Rev5 §8.7.1 Branch D will block the new modules

Branch D declines off-topic questions, explicitly listing “other universities” as off-topic. That was correct when the only content was MAJU's own website. It stops being correct the moment Layer 4 goes live. The entire scholarships module is about other universities; the jobs module is about companies. “Show me fully funded scholarships in Germany” is exactly the query §8 Layer 4 promises to answer, and Branch D as written declines it. Deployment decision (Rev 4): the whole revised prompt in §8.7.6 ships as one block at Phase 3, including the widened Branch D — not split across phases.

Rev 3 said this “will not surface until Phase 9”, which was inconsistent with §8.7.6 being a single unified prompt that Phase 3 needs for its confidence tiering. Maintaining two prompt versions to keep Branch D narrow until Phase 9 costs more than it saves. Shipping the widened branch early is harmless. Before Phase 9 there is no scholarship or job content in the vector store, so a query like “show me scholarships in Germany” retrieves nothing, lands in the low-confidence tier, and produces the no-info response or a query handoff. Slightly less clean than an explicit “I only handle university questions”, but not wrong — and it corrects itself automatically once the content exists. Add the case to the golden set so the wording is protected from drifting back. Revised scope boundary: in scope = the university's own information, plus scholarships, jobs and events held in the system's own database. Out of scope = general world knowledge, other universities' admissions processes when they are not part of a stored scholarship record, and anything unrelated to study or careers.

Rev5 §8.7.2 Hardcoded identity and personal data

Two problems in one: Branch F hardcodes a real faculty member's name and email as a worked example. This means a real person's contact details are permanently embedded in the prompt, and the model may treat that address as a default recipient when the actual recipient is unclear. Replace with a generic instruction: resolve the recipient from context or history, and use a placeholder when neither supplies one. “MAJU” and “Muhammad Ali Jinnah University” are hardcoded throughout. This conflicts with §11 — a redeployment for another university would ship with MAJU's name in every branch of the prompt, and with MAJU faculty contact details inside Branch F. Template these as variables filled from the same config that drives the rest of the branding.

Rev5 §8.7.3 The consistency rule can preserve hallucinations

The current rule states that the assistant must never contradict what it said earlier, and that reusing a fact or email from a previous turn is not inventing. The intent is right — students do ask follow-ups, and refusing to repeat something you just said is bad behaviour. But as written it applies to anything the assistant previously said, including something it fabricated. If turn 1 produced a wrong fact, turn 3 is instructed to repeat it, and the consistency rule actively protects the error for the rest of the conversation. Fix: scope consistency to facts that originally came from retrieved context, not to everything in the transcript. Repeating a previously- retrieved fact is correct; repeating an unsourced one is not.

Rev5 §8.7.4 No middle confidence tier

§4.2 specifies three tiers — confident, ambiguous (ask a clarifying question), low-confidence (offer query handoff). The current prompt is binary: answer, or return the no-info fallback. The middle branch does not exist. Note on where the tier decision is made: the tier comes from the retrieval score computed in rag.py , not from the model's self- assessment — models are unreliable judges of their own confidence. The prompt's job is only to behave correctly in the ambiguous tier, which means asking one specific clarifying question rather than guessing between candidates or punting straight to escalation.

Rev5 §8.7.5 Context is an unlimited blob

{context} is injected as one block with no per-chunk separators. Two consequences: the model cannot tell where one source ends and the next begins, and there is no rule for what to do when two chunks disagree — a stale fact page and a current one both retrieved, for example. Fix: number and delimit the chunks, and add an explicit rule for conflicts — prefer the chunk carrying a more recent date; if neither is dated, say that sources disagree and give the office contact rather than silently picking one.

Rev5 §8.7.6 Optimized prompt

Changes from the current version are marked. Everything not marked is carried over from the existing prompt, which was already correct. CONTEXT FROM {UNIVERSITY_NAME} SOURCES (applies to the current question only — earlier turns are in the chat history above) {context_numbered} # CHANGED: numbered + delimited, e.g.

...chunk text...

[2 | source: admissions FAQ | updated: unknown]

CURRENT QUESTION: {question} USER TYPE: {user_type} # NEW (Rev 5): guest | student

RETRIEVAL CONFIDENCE: {confidence_tier} # NEW: high | medium | low, from rag.py HOW TO ANSWER — decide which type this is, then follow that branch: A) GREETING / SMALL-TALK ("hi", "salam", "aoa", "thanks", "how are you") Respond warmly in 1-2 sentences as {UNIVERSITY_SHORT} Assistant and invite a question. Ignore the context. B) IDENTITY ("who are you", "what model are you", "are you ChatGPT") You are {UNIVERSITY_SHORT} Assistant, the virtual helpdesk for {UNIVERSITY_NAME}. Do NOT name any AI model, company, or technology. 1-2 sentences. C) META / ABOUT THIS CONVERSATION "what did I just ask", "mne abhi kia kaha", "which program are you discussing", "summarize our chat", "tell me more", "explain that again", "translate your last reply" Answer from the chat history. IGNORE the context block — retrieval may have pulled unrelated chunks. If there is no prior conversation, say so warmly. D) OUT OF SCOPE # CHANGED: scope widened IN SCOPE: {UNIVERSITY_NAME} information (admissions, fees, programs, courses, faculty, schedules, campus, contact, policies) AND scholarships, jobs, and events held in this system. OUT OF SCOPE: general world knowledge, coding help, math, weather, opinions, and other universities' admissions processes that are not part of a stored scholarship record. IF USER TYPE IS guest: narrower scope — {UNIVERSITY_NAME} information, INTERNAL # NEW (Rev 5) scholarships, campus events, and programs only. Jobs and external scholarships are out of scope for guests. For out-of-scope questions, decline politely and steer back in 1-2 sentences. Do not answer from your own knowledge. E) IN-SCOPE QUESTION

- Follow-ups ("and its fee?", "or fee?", "what about BSCS?"): resolve pronouns and missing subjects from the chat history first, then answer using BOTH context and the topic from history.
- If the context looks unrelated to the topic being followed up on, DO NOT switch topic. Say what the context covers about the requested topic, or say you don't have details for that specific one.
- Facts come from the context. A fact you stated earlier IS reusable — but only if # CHANGED it originally came from retrieved context. If you cannot tell where an earlier figure came from, treat it as unverified: repeat it with "as mentioned earlier, though please confirm with the office" rather than asserting it as fact.
- NEVER produce a new fee, deadline, email, phone number, course, faculty name, or policy that appears in neither the context nor the history.
- If two context chunks disagree, prefer the one with the more recent date. If # NEW neither is dated, say the sources differ and give the relevant office contact.
- Full answer in context → answer directly.
- Partial answer → give what's covered, briefly note what's missing. Do not refuse over one missing detail.
- No specific answer but a relevant office contact is present → share the contact rather than falling back to "{no_info}".
- Personal issue or complaint (registry error, payment problem, portal issue): # CHANGED (Rev 5)
IF USER TYPE IS student → tell them they can register an official ticket in the student portal under the relevant department. IF USER TYPE IS guest → they have no portal account. Give the admissions office contact instead. Never tell a guest to log in or file a ticket.
- IF USER TYPE IS guest, orient the answer to someone deciding whether to apply # NEW (Rev 5)
here — same warmth, but they are evaluating the university, not operating inside it. Do not assume enrolment, a student ID, or portal access.
- If RETRIEVAL CONFIDENCE is medium: ask ONE specific clarifying question instead # NEW
of guessing. Name the candidates you found — e.g. "Do you mean the fee submission deadline or the course registration deadline?" Do not ask a vague "can you be more specific?", and do not escalate at this tier.
- If neither context nor history can answer, reply exactly: {no_info}

F) DRAFT / COMPOSE ("draft an email", "email likh do", "write a leave application")

- Write the finished piece, not instructions for writing it.
- For an email: salutation, body covering the stated purpose, polite closing.
- Resolve the recipient and their address from the chat history or context. If # CHANGED

neither supplies one, use [recipient name] and [recipient email] as placeholders. Never supply a name or address from your own knowledge.

- Include only details the student gave or that appear in context/history. Use short placeholders like [your name] or [dates] for genuine unknowns.
- Do not refuse over missing minor details — draft with placeholders.

ALWAYS:

- {lang_hint}
- Bold **email addresses, phone numbers, fee figures, deadlines/dates.**
- Roman Urdu: keep it natural and student-friendly. Use English loanwords directly ("fee", "admission", "department", "office", "course") rather than formal Urdu equivalents ("akhrajaat", "dakhla", "shoba").
- Quote fees, dates, emails and phone numbers EXACTLY as they appear in the context.
- Start with the answer. No "Sure!", "Of course!", "Here is", "Based on the context", and no sign-offs.
- Never mention sources, source numbers, "[1]", or a "Sources:" section.
- Tone: warm, helpful, professional — a friendly student services officer. Be concise.

Bullet lists only when the answer is genuinely a list. ANSWER: On chunk numbering and the no-citation rule: these are not in conflict. The numbering exists so the model can distinguish sources internally and so the grounding check (§13.3) can match an answer back to the chunk it came from. The student still sees a clean answer with no bracket references. Measure before adopting. This prompt is a proposal, not a drop-in replacement. It changes classification behaviour in Branch D and adds a new tier to Branch E — both of which can shift results in ways that are hard to eyeball. Run it against the golden set alongside the current prompt and keep whichever scores better. That is exactly what §13.2 exists for.

Rev5 §8.8 The retrieval stage

Sections 8.4 and 8.7 cover what happens after chunks are retrieved. This section covers what happens before — where two real gaps sit, one of them a genuine bug rather than a missing improvement.

Rev5 §8.8.1 Follow-up questions reach retrieval unresolved

This is the most important item in this section. The prompt (§8.7, Branch E) resolves pronouns and missing subjects from chat history — “or fee?” after a BSCS turn is understood as “BSCS fee”. That works, but it happens at the generation stage. Retrieval never sees it. So the sequence today is: Student: "or fee?" → retrieval runs on the literal string "or fee?" → returns whatever weakly matches two words with no subject → prompt resolves the topic from history and does its best with chunks that may not be about BSCS at all. The prompt is compensating for a retrieval failure. Sometimes it succeeds, because the fact is still in the chat history from an earlier turn. But when the answer needs a chunk that was never retrieved, no amount of prompt work recovers it — the information simply is not in context. This is very likely a live source of wrong or incomplete answers on follow-up questions, and it would not be obvious from testing single-question flows. Fix — resolve the query before retrieval, not only in the prompt:

Student: "or fee?" → rewrite step: read chat history, produce a standalone query → "BSCS tuition fee" → retrieval runs on the rewritten query → prompt receives relevant chunks AND the history Implementation options, cheapest first: Heuristic — if the query is very short, lacks a subject noun, or starts with a conjunction ("or", "aur", "and", "what about"), prepend the topic from the previous turn. Zero cost, catches the common cases. Small LLM rewrite call — one cheap call producing a standalone question from history plus the current query. More reliable, but adds a call per follow-up. Cache aggressively; many follow-ups repeat. Start with the heuristic and measure. The golden set (§13.2) must include multi-turn sequences for this to be testable at all — a

set of standalone questions will not reveal this bug. Scheduled at Phase 3 (Rev 4), alongside the prompt revision — both change how a follow-up question is handled, and testing them together against the same golden set is more informative than testing them apart.

Rev5 §8.8.2 Query expansion for vocabulary mismatch

Students write “fees kitni hain”; the knowledge base says “tuition fee structure per credit hour”. Semantic search bridges some of this gap, but not reliably — and less reliably for Roman Urdu, since the embedding model is predominantly English-trained. The existing hybrid keyword layer helps for exact terms and is one reason the current pipeline works as well as it does. Beyond that, options are: a small synonym map for the terms that actually recur (fee/fees/challan/dues, admission/dakhla, transcript/degree), or an LLM expansion step generating alternate phrasings before retrieval. Do the synonym map first. It is free, deterministic, testable, and the failed-question log (§4.3) tells you exactly which terms to put in it — rather than guessing at vocabulary students might use.

Rev5 §8.8.3 Chunking and k were never reviewed

Two parameters that shape retrieval quality more than most algorithm choices, and neither has been examined: Chunk size, overlap, and heading anchoring. `chunk_elements()` was written for MAJU's prose pages. The retrieval-dilution bug fixed earlier in the project is the kind of problem that usually originates in chunking rather than in the retrieval algorithm — if a fee table is split across two chunks, no retrieval strategy recovers the whole table. How many chunks are retrieved (`k`). Too many and the relevant chunk is buried in noise — that is dilution. Too few and answers come out incomplete. This is a single number, and it is one of the highest-leverage things to tune against the golden set. Neither of these should be changed on intuition. They are exactly what the golden set exists to settle.

Rev5 §8.8.4 Priority

Item When Why Multi-turn cases in the golden set Phase 1, with the golden set Without them, 8.8.1 is invisible Follow-up query rewriting (heuristic) Phase 3, with the prompt revision Likely a live bug, not an enhancement `k` tuning Same pass One number, measurable, high leverage Synonym map Once the failed-question log has data The log tells you which terms matter Chunking review Only if the golden set shows retrieval failures Do not rewrite chunking on a hunch

Rev5 §9. Admin: content and source management

Rev5 §9.1 Manual entry

Manual add, edit and delete for all three modules. One admin panel, three tabs, same shape. This is a fallback, not the primary method. Automation is the plan. But if a source goes down or starts blocking, being able to add entries by hand keeps the system usable. It does not make the project “manual” — it makes it robust. UI design: a new top-level “Content” item in the admin sidebar with four sub-tabs — Jobs, Scholarships, Events, Sources — following the

existing tabbed layout pattern. Each module tab: status filter (All / Pending / Approved / Rejected) above the existing AdminTable + Pagination + LoadingSkeleton + EmptyState structure Row click opens the existing right-side slide-over drawer pattern, but with a schema-driven editable form and Approve / Reject / Save / Delete in the footer “Add Manually” opens a modal with the same form, saving directly as `status=approved` Build one generic component and one generic slice parameterized by type, rather than three near-identical copies. The spec already describes these as “one panel, three tabs, same shape” — the implementation should match that description. A single generic GET `/api/admin/listings?type=...` route mirrors this server-side.

Rev5 §9.2 Source management — add a source by URL

Sources live in the database, not in code. This is the single best structural decision in the plan. Two modes: One-time scrape — paste URL → “Scrape Now” → extracted data returns in a pre-filled editable form → admin edits → approves → saved. Recurring source — paste URL → saved to the sources collection with type and frequency → joins the scheduled cron automatically with hash detection. Why this matters so much: Adding a new company or portal requires no deploy A source that breaks or starts blocking gets paused; everything else keeps running You can launch with five sources and grow — no pressure to verify thirty before starting In a demo you can add a live URL in front of the examiner and scrape it. That is the strongest single moment available in the whole presentation, and it demonstrates the entire pipeline in about thirty seconds. It also dissolves several open questions. “Will Rozee allow scraping?” and “which companies can we scrape?” stop being build-time architecture decisions and become runtime configuration. Whatever works gets added; whatever does not stays paused. Seed sources and admin-added sources are the same thing. The initial known set is not hardcoded into application logic — it is loaded into the same collection once at deploy time via a seed script, mirroring the existing `seedAdmin.js` pattern. After that there is no

distinction: same schema, same scraping, same health monitoring, same auto-pause. The only difference is how the row arrived. Schema: Field Notes url type job / scholarship / event name frequency daily / weekly last_run last_hash status active / paused / failing consecutive_failures entries_pulled_last_run

Rev5 §9.3 Cross-service architecture

The Source record and resulting listings live in MongoDB (Node's domain — Layer 1 matching queries Mongo directly). Fetching, hashing and extraction happen in Python (where the scraper plumbing and LLM clients live). Every scrape crosses the boundary. Design rule: Node owns all state and business rules. Python is a stateless executor that fetches, extracts and reports back. This keeps frequency interpretation and failure counting in one place rather than duplicated across two languages. One-time scrape (admin clicks Scrape Now):

1. Admin submits URL + type → Node route
2. Node sanity-checks the URL, then makes a synchronous call to a new Python endpoint with an internal-secret header — the admin is

waiting on screen, so this is request/response, not a queued job

3. Python fetches (SSRF validation is authoritative here — Python is the only side that sees the redirect chain), hashes, extracts, returns

structured JSON plus hash

4. Node creates the listing as status=pending and shows the pre-filled editable form

New requirement: no Python endpoint currently validates an incoming internal-secret header — the existing pattern only runs Python→Node. This new endpoint needs a FastAPI dependency validating it, since it is a privileged action that must not be reachable unauthenticated. Recurring source (background):

1. A Python-side scheduler wakes on a fixed interval (every 30–60 minutes). It does not decide what is due.
2. It calls a Node endpoint that computes which sources are due from frequency and last_run — keeping “what counts as due” in Node

next to the schema, so the admin UI can display next-run information without reimplementing the logic

3. For each due source, Python fetches, hashes, and extracts if changed
4. Python reports back via a webhook-style POST using the existing internal-secret pattern — no new auth pattern needed in this direction
5. Node updates last_run , last_hash , increments or resets consecutive_failures , auto-pauses at 3 failures, and creates the pending

record Why this split: it reuses the one cross-service auth pattern that already exists for the background direction, and introduces only one genuinely new pattern for the synchronous call — rather than inventing two. Python's scheduler stays a simple ask-for-work / do-it / report- back loop with no business logic.

Rev5 §9.4 Source health monitoring

Per source: last successful run, entries pulled, consecutive failures, current status. Auto-pause after 3 consecutive failures and flag to the admin. Without this you end up with thirty sources and no idea that five have been broken for months. Silent failure in a scraping system looks exactly like “that site had no new listings.” Sources tab UI: - Table columns: URL (truncated), Type badge, Name, Frequency, Last Run (relative), Status badge (green/amber/red using existing color tokens), Consecutive Failures, Entries Pulled - “Add Source” modal: URL, Type dropdown, Name, and a One-time vs Recurring choice — Recurring reveals a Frequency dropdown - One-time triggers the synchronous scrape with a loading state (“Scraping... this can take up to 15 seconds”), then transitions into the pre-filled edit form - Recurring just saves the row; the scheduler picks it up - Row actions: Pause/Resume, Delete - Row click opens a drawer with health details plus a “Run Now” button to force an immediate scrape without waiting for the schedule — useful for testing, and the mechanism behind the live demo moment

Rev5 §9.5 Review queue

Everything scraped lands as status=pending . Admin approves, rejects or edits. Manual entries go straight to approved . Every record carries source (scraped/manual) and status . The status values are pending , approved , rejected , plus the lifecycle states from §6.5: expired and delisted . Merge review (added in Rev 4). The queue has a filter for merge events, showing what

was merged into what and the similarity score that triggered it, with an unmerge action. See §8 Layer 2. Why source matters: it prevents a re-scrape from overwriting manual entries, it makes debugging possible, and it lets you state honestly in the report exactly how much of the data is automated. Codebase note: the existing `auditAdminWrites` middleware auto-logs any non-GET admin route mounted under `adminRoutes.js` — new source routes get audit logging free if placed there. Its `targetId` extraction needs extending to recognize a source key.

Rev5 §9.6 Security — SSRF validation

The admin panel accepts arbitrary URLs and fetches them server-side. That is an SSRF risk: a malicious or careless URL could reach internal network services. Allow only http and https Block internal ranges: localhost , 127.0.0.0/8 , 10.0.0.0/8 , 172.16.0.0/12 , 192.168.0.0/16 , 169.254.0.0/16 Block non-standard ports Re-validate after every redirect — a permitted URL can redirect to an internal one Small check, real vulnerability, worth writing up. Worth unit-testing (§13.2) — pure function, no mocking, and a silently broken validator is exactly the kind of bug nobody notices.

Rev5 §10. Notifications and email

Rules Digest, not instant. A scraper pulling 40 jobs a week must not produce 40 emails. The student picks daily or weekly. Instant per-match emails are the fastest way to get unsubscribed. Strong matches only. Set a score threshold. Send “you’re eligible” items, not “maybe” items. Deadline reminders are separate and immediate. “The scholarship you saved closes in 7 days” is the single most valuable email this system can send — more valuable than any new-match notification, because the student already decided they wanted it. Saved-item change alerts. “The IELTS requirement on a scholarship you saved changed from 6.5 to 6.0.” This is what makes change detection visible to users rather than invisible backend work. Unsubscribe link and frequency control on every email. Delivery Gmail SMTP was fine for FYP-1 issue notifications — low volume, transactional, one recipient at a time. It will not work for bulk digests: daily sending limits and spam filtering. Move digests to a transactional provider (Resend or Brevo — both have adequate free tiers). Send from a background job, never inside the request cycle. Codebase note: `notify.js`’s SMTP transporter and HTML template helper are reusable, but `notify()` is single-recipient/single-event and tied to a closed `Notification.type` enum. Digesting needs a new function, a new template, and a new enum value — not a drop-in reuse. The scheduler built for §4.1 is what triggers this.

Rev5 §11. Deployment and rebranding

Single-tenant. One university per deployment, rebranded each time. Config-driven branding from day one: `UNIVERSITY_NAME` `UNIVERSITY_SHORT` `EMAIL_DOMAIN` `STAFF_EMAIL_PATTERN` `LOGO_URL` `PRIMARY_COLOR` `SUPPORT_EMAIL` Do this now. Later it means hunting “MAJU” through fifty files. Doing it while the new modules are being written costs almost nothing. Knowledge base setup for a new deployment: the existing `scraper.py` already scrapes MAJU pages for the chatbot. Make it admin-facing — a new admin pastes the university’s website URL, the scraper runs, the vector DB gets built. Roughly 80% of this exists already. Turning it into an admin flow is what makes same-day deployment realistic. Startup config validation (§13.1): `server/config/db.js` currently only logs a connection error and keeps running. For a product redeployed per-university, the same class of misconfiguration recurs at every deployment. Fail loudly and immediately instead of running half-broken.

Rev5 §12. Build order

Verification comes first. The plan rests on assumptions about websites nobody has tested, and there is no Apify fallback to absorb the failures.

Phase Work Why here 0 Verify every candidate source: robots.txt, ToS, anti-bot, JS-rendered or not. Resolve the Pydantic v1/v2 decision. Test Crawl4AI locally on 3–4 sites. Everything downstream assumes these answers. The Pydantic decision gates the extraction library. 1 Unified profile schema + CV parsing + hard- rule matching + golden-set evaluation Foundation for everything else. The golden set is here because Phase 3 changes the prompt and needs a baseline to change against. 2 Scheduler infrastructure + SLA escalation + issue feedback The scheduler is needed three more times later. Building it early with a small, self- contained feature de-risks it. 3 Chatbot→query handoff + failed-question log + revised RAG prompt (§8.7.6) deployed whole + follow-up query rewriting (§8.8.1) Cheapest high-value work available — the confidence score already exists and is being discarded. Forms the self-improving loop. Score every change against the Phase 1 golden set. 4 Source management + review queue + manual entry Needed before any pipeline is useful 5 Scholarships — full pipeline No legacy code to reconcile. Build the pipeline clean, once. 6 Jobs — reusing the proven pipeline; retire `Job.jsx` per §2.3 Pipeline is now tested; this is schema + sources + UI 7 Events — pending the volume/format check Smallest scope 7b Listing lifecycle (§6.5) + delisting detection (§7.4) Needs all three pipelines producing data before expiry and delisting are meaningful 8 Embeddings matching + ingestion dedup + LLM explanations Layer on once data flows 9 RAG over new content + skill gap analysis Reuses existing

architecture 10 Alerts and digests Needs data and matching in place 11 PWA See §14 Two changes from Rev 2 worth explaining: Section 4 improvements moved from Phase 8 up to Phases 2–3. They were at the end of the list. They are the items that actually make the system better for students, and the chatbot handoff in particular is unusually cheap because rag.py already computes the confidence score and throws it away. Phase 2 also builds the scheduler that Phases 4, 7 and 10 all need — building it early against a small feature is lower risk than building it under pressure later. Scholarships before Jobs. Rev 2 had Jobs first on the grounds that careers pages are simpler than scholarship portals. That is true of the pages, but Jobs also carries a 1300-line legacy implementation to retire, while Scholarships has none. Building the pipeline where there is no legacy code to reconcile is the cleaner first pass; Jobs then becomes schema plus sources plus UI on a pipeline that already works. The engineering additions in Section 13 are not a phase. Bucket 1 lands alongside whatever is being built. See §13.

Rev5 §13. Engineering additions — bucketed

Twenty items came out of the audit rounds. Each is individually cheap and genuinely valuable. Twenty individually-cheap items are not collectively cheap, and “can land alongside any phase without blocking it” is how scope creep starts. They are therefore sequenced rather than listed. Nothing here is rejected on merit — this is about when, not whether.

Rev5 §13.1 Now — build alongside core work

These are cheap, one-time, and get materially more expensive if deferred. Shared student-side component library. Extract card, filter chip, search bar, empty state, loading state once — reused across the rebuilt Jobs, Scholarships, Events and Profile pages. Timing is why this is here. The admin side already has this pattern; the student side does not — the current pages hand-roll their own versions, with at least three different loading-state implementations between them. Since all four pages are being rebuilt from scratch

anyway, this is the cheapest possible moment. Doing it later means writing the same components three times and then extracting them. Security cluster — roughly one evening for all four: helmet.js — no HTTP security headers exist today. One dependency, a few lines, a real item for the report’s security section. CORS lock-down — verify the config is restricted to the actual client origin. Single-tenant, single-frontend: there is no reason to accept cross-origin requests from arbitrary hosts. Auth rate limiting — express-rate-limit is already a dependency; verify it is actually applied to login and OTP-resend, and wire it up if not. These are natural brute-force targets. Guest chat rate limiting (added Rev 5) — the guest chatbot is the only unauthenticated endpoint in the system that calls a metered LLM API, which makes it the largest free-tier exposure the project has. IP-based limiting plus a per-session message cap. See §19.6. Login lockout — lock an account after N failed attempts in a short window. LoginEvent already records every attempt with a success flag and timestamp, so the data needed already exists and is currently unused. Close to the cheapest security improvement available in this codebase. Startup config validation. Fail fast with a clear message when a required env var is missing or invalid. Since the product is redeployed per-university, this misconfiguration class recurs at every deployment. ChromaDB backup. Scheduled snapshot of the vector store, or a periodic JSON export, stored outside the live working directory. Why now: chroma_db is gitignored and has no backup path. A full-rebuild bug, a bad admin edit, or disk corruption silently wipes the entire chatbot knowledge base with no way back. This is a current single point of failure, not a future one.

Rev5 §13.2 Next — immediately after core

These need data and working features to be meaningful. Golden-set evaluation — build this in Phase 1, not in this bucket. Moved in Rev 4. Rev 3 placed the golden set in this “next” bucket while also requiring every prompt change to be scored against it — but Phase 3 changes the prompt. The set would not have existed when it was first needed, which contradicts this document’s own “measure before you change” principle (§2.4). It is one day of work and it gates Phase 3, so it belongs in Phase 1. It is described here because everything else in §13.2 and §13.3 depends on it. A fixed set of roughly 20–25 representative questions with known-good expected answers — English, Roman Urdu, and mixed, covering both the existing MAJU knowledge base and the new content. Spot-check whenever prompt, retrieval or chunking logic changes. The set must include multi-turn sequences, not only standalone questions. The follow-up retrieval gap in §8.8.1 is invisible to a set of independent questions — it only appears when a short elliptical follow-up depends on a chunk that was never retrieved. Include several two- and three-turn sequences. This gates prompt changes as much as retrieval changes. Prompt edits feel free — no dependencies, no deploy, just text — so they get made casually and often. But every edit improves some questions and quietly breaks others, and with a six-branch classification prompt (§8.7) the failure mode is usually misrouting rather than a visibly wrong answer, which is harder to notice by hand. Any change to the prompt should be scored against the set before and after. Treat the prompt as code that needs a regression test, not as copy. Why this comes before the retrieval improvements in §13.3: rag.py has already had real bug fixes (retrieval dilution, elliptical follow-up handling), which shows the pipeline gets iterated on. But there is no reference set,

so a fix for one question can silently regress another. This is also exactly the reasoning used to defer LiteLLM (§15) — do not replace working, recently-fixed code without a way to detect regression. The same argument applies to inserting a re-ranker into the retrieval path. On tooling: RAGAS produces standardized metrics and gives concrete numbers for the report's evaluation section. Note its cost, though — most RAGAS metrics use LLM-as-judge, so 25 questions across 4 metrics means a substantial number of calls against the same free tier the whole system runs on. Start with simpler manual scoring and add RAGAS once the set has proven useful; the golden set is the valuable part, the tooling is secondary. Targeted unit tests. Not a full suite — four functions that are pure, deterministic, and high-stakes: Function Why it's on the list SSRF validator (§9.6) A broken check is a security hole nobody notices Hash change detection (§7.2) Broken one way burns the API budget; broken the other way stops updating data. Both silent. Hard-rule matching (§8 Layer 1) Everything downstream depends on its output CGPA conversion (§6.2) Wrong answers look plausible Zero automated tests exist today. These four are the cheapest to test — no mocking of LLMs, databases or network — and the most valuable to prove correct. Good evidence for the report's evaluation section.

System health / pre-demo check. One admin screen pinging every external dependency — LLM providers, Mongo, ChromaDB — with green/red status. The demo checklist (§17) already warns about third-party failure; a one-click check before walking into the exam room is cheap insurance. Near-miss bucket. Surface borderline-ineligible listings with what is missing: "IELTS 6.5 required, yours is 6.0 — a retake before the 15 January deadline would qualify you." Cheap because Layer 1 already records the gap data. Genuinely useful because it tells students what to fix rather than only what they qualify for today. Department-scoping middleware. Extract the repeated "does this staff user have a department" guard, currently duplicated across roughly six functions in `issueController.js`. FYP-2 adds more department-scoped resources; without this the clause gets copy-pasted again each time.

Rev5 §13.3 Later — after FYP-2 core is stable

Real value, but each depends on something that does not exist yet, or addresses a problem that has not appeared yet. Note on the two RAG-quality items below. They are gated on a prerequisite, not on the calendar. The blocker is the golden set (§13.2) — nothing else. Once the golden set exists, both move out of this bucket and become immediate work, regardless of which phase the project is in. Do not read "Later" here as "low priority": the current RAG pipeline is architecturally standard but has no quality controls, and these two are the quality controls. They are placed here only because changing retrieval without a baseline means being unable to tell an improvement from a regression. Sequence: golden set → confidence surfacing (already required by §4.2, and the score is currently computed and discarded) → re-ranking → grounding check. Measure each change against the set. Cross-encoder re-ranking (§8 Layer 2.5). Blocked on the golden set. Inserting a re-ranker into a recently-fixed retrieval path without a baseline means not knowing whether it helped. With the set in place, this becomes a measurable change: run the set before and after, keep it if the numbers improve, drop it if they do not. The decision is made on data rather than intuition. Grounding check on chatbot answers. A cheap keyword/semantic overlap check between an answer and its source chunks, catching outright hallucination without a second full LLM call per response. Reserve an actual LLM verification call for ambiguous cases only. Same dependency as above — needs the golden set to validate. RAG answer caching. Cache identical or near-identical questions to cut API calls. Sound in principle and the same don't-recompute logic used elsewhere, but currently speculative — implement when rate limits actually bite, and use the health-check page and real usage to tell you when that is. Rev 5 note: this stops being speculative for the guest endpoint. Guest questions repeat far more than student questions — fee structure, admission dates, merit — and that endpoint is unauthenticated. Once the guest funnel (§19) exists, caching in front of it is the main defence for the most exposed part of the system, and should be promoted out of this bucket. Matching explanation audit trail (§8.6). Debugging aid, not student-facing. Cheap whenever it happens because Layer 1 records the gap data. Student data export / delete. "Download my data" and "delete my account" covering the whole profile. Completes the privacy story that §5.3 starts for CVs. Two routes, no new infrastructure.

Rev5 §14. Scope

The list is long: three pipelines, source management, a multi-layer matching engine, RAG over new content, CV parsing, email digests, saved items, application tracker, calendar view, five improvements to existing modules, plus the engineering additions. This is more work than FYP-1 contained. Must have — three pipelines · source management + review queue · unified profile · hard-rule matching · matched feed + table + detail · basic alerts · Section 4 improvements Nice to have — application tracker · calendar view · skill gap analysis · embedding dedup · LLM explanations · near-miss bucket Mobile app — reconsider. A React Native app for three roles with push notifications and offline support is a project in itself. A responsive PWA gets roughly 90% of the benefit: installable, push-capable, one codebase, no app store process. The months saved are better spent making scholarship matching genuinely good. A working matching engine is a stronger FYP result than a mediocre matching engine with a native wrapper around it. Recommendation: PWA now, native only if time genuinely remains.

Rev5 §15. Dropped, with reasons

Recorded rather than silently removed, so the decision is visible and can be revisited. Admin-panel-configurable LLM API keys
What it was: move provider keys out of .env into an admin-managed settings screen with encrypted storage, rotation UI, masked display, and a Python-side refresh mechanism. Why dropped: The complexity is substantial — an encrypted settings collection, a rotation interface, masking, and a cross-service refresh path. The problem it solves occurs two or three times a year, at new deployments. It also introduces a circular dependency without resolving it: if keys are encrypted in the database, the encryption key has to live somewhere, and that somewhere is .env . So the mechanism built to avoid .env still depends on .env , while adding a new security surface. Environment variables are the standard answer for secrets and they are the right answer here. Editing one file during a redeploy is acceptable — deployment already involves configuring a database URI, an SMTP account, and branding values. Revisit if: deployments become frequent enough that redeploy friction is a real cost. LiteLLM refactor
What it was: replace the hand-written multi-key rotation and provider fallback in rag.py with LiteLLM's unified interface. Why dropped: the existing logic has been debugged recently. Replacing working, recently-fixed code with a library carries real regression risk for marginal benefit. The Rev 2 spec reached this conclusion itself and it is correct. Revisit if: provider handling needs significant extension anyway, or after FYP-2 is stable. Matching engagement feedback loop
What it was: track saves, clicks and applications on matched items as a ranking signal. Why dropped: the Rev 2 spec already flagged it as future work. It adds a data-collection surface and a re-ranking mechanism, and it needs real usage data to be worth anything — which does not exist yet. Revisit after: the system has real students using it for a meaningful period.

Rev5 §16. Open items — verify before building

Nothing here is settled. Do not architect around any of it.

1. Pydantic v1 vs v2 — resolve in Phase 0. requirements.txt pins 1.10.13 . Instructor primarily targets v2, and moving to v2 may break ChromaDB or langchain pins. This gates the extraction library choice, so it is a blocker rather than a note.
2. Source viability. For every candidate site: robots.txt rules, terms of use, anti-bot presence, JS-rendered or server-rendered. Not checked for a single site yet. The entire pipeline plan rests on this.
3. Rozee.pk. Check robots.txt and ToS. It is the main source of local Pakistani job listings, so it matters — but only if permissible and feasible.
4. Which Pakistani companies use an ATS. Test predictable board URL patterns against a company list. Expect a low hit rate; get the real number rather than guessing.
5. Remote job board API coverage. Verify current free-tier status, rate limits, and whether Pakistan-relevant roles actually appear.
6. Free LLM tier limits. Check the providers' own pricing pages — published third-party numbers vary widely and change often. Design around the faster/cheaper tier, not the flagship, whose free limits are typically too low to build on.
7. MAJU event volume and format. How many events per semester, and text or poster images? Determines whether the events pipeline is worth automating at all.
8. Crawl4AI on the dev machine. It runs Playwright, which downloads browser binaries and needs meaningful RAM. Confirm it runs comfortably on 8 GB alongside everything else before committing the pipeline to it.

Rev5 §17. Demo checklist

Cached snapshot ready. Never depend on live fetching during a presentation. A site being down, a rate limit, or campus wifi can end the demo. Health check run immediately before starting — all dependencies green. Live source add. Paste a URL in front of the examiner, scrape it, show the result. Strongest single moment available. Unfiltered vs matched. Show the raw list, fill the

profile, show the same data ranked with eligibility reasoning. Proves the matching engine in about ten seconds. Duplicate merge. Prepare a case where one job arrives from multiple sources and gets merged — the detail page's "found on N sources" display (§8 Layer 2) is what makes this showable rather than invisible. Chatbot failure → query handoff. Ask something the bot cannot answer; show the one-click escalation into a department query. SLA escalation. Show an overdue issue being escalated.

Rev5 §18. Summary of decisions

Area Decision Deployment Single-tenant, rebranded per university. No multi-tenancy work. Data acquisition In-house scraper only, free tiers. Apify retired, no hybrid fallback. Sources Stored in DB, added via admin panel by URL. Seed and admin-added treated identically. Scrapers LLM extraction with Pydantic schemas. No CSS-selector scrapers. Fetching Crawl4AI, self-hosted. No credit-metered scraping services. Cost control Hash-based change detection. Only changed pages hit the model. Models Free hosted tier with failover for public pages; cloud with PII stripping for CVs. CV privacy PII stripped before external processing; raw file deleted after parsing; student informed. Profile One unified profile in four sections. Profile access Soft banner. No hard gate — except email alerts. CV flow Upload → strip → extract → editable pre-filled form → confirm. Never silent save. UI Matched feed primary, browse table secondary, detail + redirect link. Matching Cascade: hard rules (3-state) → embeddings → LLM explanation, content- hash cached. Matching timing Hybrid — Layers 1–2 on request, Layer 3 lazy + cached. RAG prompt Five fixes to the existing prompt (§8.7); templated for rebranding; validated against the golden set before adoption. RAG retrieval Follow-up queries rewritten before retrieval, not only in the prompt (§8.8.1). k and chunking tuned against the golden set, not on intuition. Prompt deployment The full revised prompt ships as one block at Phase 3, widened Branch D included. No split versions. Golden set Built in Phase 1, because Phase 3 changes the prompt and needs a baseline. Listing lifecycle Expired and delisted are statuses, never deletions. Saved items keep history with a badge. Delisting Detected by per-source listing-set comparison, guarded by successful- run and two-consecutive-absence rules. Search Keyword only. Semantic querying belongs to the chatbot — table for lookup, chatbot for questions. Staff role No access to Jobs or Scholarships. May submit Events for their own

department into the review queue. Merges Visible to students as source count, reviewable and reversible by admin, similarity score logged. Past events Retained in a separate tab, excluded from feed and alerts. Guest model A prospective applicant with their own goal, not a student with features disabled. Guest content Internal scholarships, campus events, programs, chatbot. No jobs, no external scholarships. Scholarship scope internal / external flag. External is registered-students-only. Guest prompt {user_type} branching; portal-ticket advice replaced with office contacts. Guest protection IP rate limiting, per-session cap, and RAG caching promoted to the guest endpoint. Email Digest-based, thresholded. Transactional provider for bulk. Branding Config-driven from day one. Mobile PWA over native React Native. Rework policy Full replacement of Apify-dependent code. Build → verify → cut over → delete. Commit hash recorded first. Build order Scholarships before Jobs. Section 4 improvements early, not last. Engineering additions Bucketed into now / next / later. Three dropped with reasons.

Rev5 §19. The guest experience

New in Rev 5.

Rev5 §19.1 A guest is not a limited student

Rev 4 modelled the guest as a student with most features switched off. That framing is wrong, and it produces a worse product for both parties. A guest is a prospective applicant deciding whether to apply here. They ask different questions: Registered student asks Guest asks When is my fee due? What is the fee structure? What's the improvement-exam rule? What was last year's merit for BSCS? Where do I get a transcript? When does admission open? Who is my course advisor? What is campus life actually like? A student is operating inside the university. A guest is evaluating it. Once that is clear, the guest experience has a purpose of its own: help the applicant decide, and make MAJU look like the right choice. This is also the first part of the product that serves the university's own commercial interest rather than only the student's — it handles admissions enquiries and engages applicants. That distinction is worth making explicitly in the report and in any conversation with university administration, because it is a revenue-side argument rather than a cost-side one.

Rev5 §19.2 What a guest sees

Chatbot — full access, scoped to public information. This is the guest's main tool and where most of their questions land. MAJU's own scholarships — yes. External scholarships — no. Add a scope flag to the scholarship schema: scope: internal | external

internal — MAJU's own merit awards, need-based aid, sports scholarships. Visible to guests. external — DAAD, Chevening, CSC, and the rest of §6.2. Registered students only. Why external scholarships must be hidden from guests: they point a

prospective applicant away from MAJU. Showing a school-leaver a list of funded programmes abroad, on MAJU's own admissions page, works directly against the purpose of that page. Internal scholarships do the opposite — they are a reason to apply. Campus events — yes. Sports galas, tech fests, society events, convocations. These are already internal by nature. This matters more than it looks. An eighteen-year-old choosing a university is influenced by campus life at least as much as by fee tables, and a live feed of what actually happens on campus communicates that better than any brochure page. It is also content the university already produces and currently does nothing with. Jobs — no. Irrelevant to someone who has not enrolled. (An argument exists for showing graduate placement outcomes as a recruitment signal, but that is a different feature from a job board and is not in scope here.) Programs — yes, if program data is available in structured form. A guest deciding between BSCS and BSSE benefits from a comparison far more than from a chatbot conversation. A note on browse-only: the guest sees the browse table, never the matched feed — matching requires a profile, and a guest has none. This is the same soft-banner principle as §5.4, one level down: show the value, then ask for the sign-up.

Rev5 §19.3 The guest landing page as an admissions funnel

Once the guest has a purpose, the landing page follows from it. It should not be a generic marketing page with a chat widget bolted on. Structure: Primary: the chatbot. This is what the guest actually came for — “when does admission open”, “what’s the fee”. Give it the top of the page, not a corner bubble. Below: MAJU scholarships. Concrete reasons to apply. Below: upcoming campus events. Concrete evidence of campus life. Below: programs, if available structured. Persistent: “Already a student? Log in” and a link to admission information. Context-aware follow-ups on chatbot answers. When the guest asks about a program, the answer carries relevant chips beneath it: “What’s the BSCS fee?” → answer → Scholarships available for BSCS · Admission closes 15 September · Ask about admission requirements This is a small addition with a large effect: it turns a single answered question into a path forward. It is also cheap — the data is already in the system, and the chips are driven by the same retrieval that produced the answer.

Rev5 §19.4 Prompt changes for guests

The current prompt has a live bug for guests. Branch E instructs the assistant to tell users they can register an official ticket in the student portal. A guest has no login and no portal access. Every time that branch fires for a guest, the advice is unusable. Add {user_type} to the prompt (guest | student), and branch on it: Situation Student Guest Personal issue or complaint “Register a ticket in the portal under the relevant department” Give the admissions or relevant office contact Scope (Branch D) University info + scholarships + jobs + events University info + internal scholarships + events + programs. Jobs and external scholarships are out of scope. Tone Current student services officer Same warmth, but oriented to someone deciding whether to join Query handoff (§4.2) Creates a department issue Not available — offer the admissions contact instead The query-handoff difference is important. §4.2’s low-confidence path creates a departmental issue, which requires an account. For a guest, the equivalent path is the admissions office contact — same intent, different mechanism.

Rev5 §19.5 Guest chat history carries into registration

A guest talks to the bot for twenty minutes, decides to apply, gets admitted, and registers. Under Rev 4, all of that is gone. Fix: hold guest chats against a session identifier. On registration from the same session, associate that history with the new account. Low effort, and it makes the first logged-in experience continuous rather than blank. It also preserves genuinely useful context — the questions someone asked before applying say a lot about what they will need help with afterwards.

Rev5 §19.6 Rate limiting on guest chat

This is the project’s single largest free-tier exposure and Rev 4 did not address it. Every other LLM path in the system sits behind authentication. The guest chatbot does not — it is a public, unauthenticated endpoint calling a metered API. A crawler, a script, or a single enthusiastic user can consume the entire daily quota, and the failure lands on registered students too, since they share the same provider keys. Required: - IP-based rate limiting on the guest chat endpoint, tighter than the authenticated limits in §13.1 - A per-session message cap, with a prompt to register on reaching it - Guest chat included in the health-check page (§13.2) so quota exhaustion is visible before it becomes a support problem RAG answer caching (§13.3) is more valuable here than anywhere else. Guest questions repeat heavily — fee structure, admission dates, merit, eligibility — far more than student questions do. A cache in front of the guest endpoint absorbs most of that traffic. This is a good reason to move caching up from the “later” bucket once the guest funnel exists: it stops being speculative and becomes the main defence for the most exposed endpoint in the system.

Rev5 §19.7 Guest failed-question log

Keep guest failures separate from student failures in §4.3. They are a different signal. Student gaps are missing operational content — a policy that was never uploaded. Guest gaps are missing admissions content, and they say something the university cannot easily learn any other way: what prospective applicants want to know and cannot find out. “Twenty-three prospective students asked about hostel facilities this month and we have nothing published about it” is directly actionable, and it is the kind of finding that makes the system valuable to the administration rather than only to students.

Rev5 §19.8 What this adds to the build

Item Effort Where scope flag on scholarships; events already internal Small Schema {user_type} in the prompt with guest branches Small — but fixes a live bug §8.7.6 Guest rate limiting + session cap Small §13.1, do it early RAG caching in front of guest chat Medium — promoted from §13.3 Once the funnel exists Landing page rebuild as funnel Medium Design phase Context-aware chips on answers Medium After the funnel Guest chat history migration Small With auth work Separate guest failed-question view Small With §4.3 MAJU scholarships as a structured source Medium New source type — see below On the last item: the university site is currently scraped only as unstructured chunks for the chatbot. Internal scholarships now need to exist as structured records as well, so they can be filtered, displayed in a table, and matched for registered students. Same pipeline (§7), different target pages, scope: internal set at ingestion. Worth confirming during Phase 0 that MAJU actually publishes its own scholarships in a scrapable form — if not, this is a manual-entry case (§9.1) and entirely acceptable as one.

PART B — Rev 6: Codebase Verification Addendum (original, complete)

Preserved in full from UniAssist_FYP2_Spec_Rev6_Addendum.pdf.

UniAssist FYP-2 Spec — Rev 6 Addendum Codebase verification pass — companion to Rev 5, not a replacement Project: UniAssist — Centralized AI Student Portal University: Muhammad Ali Jinnah University (MAJU) Date: August 2026 Base document: UniAssist_FYP2_Spec_Rev5.pdf

Rev6 — Revision history (appends to Rev 5's table)

Rev What changed Why Rev 6 (this document) A codebase verification pass: every specific factual claim in Rev 5 was checked against the current repository (file:line evidence). One claim in §3 is corrected. Four new findings are recorded that Rev 5's own audit did not catch, because they are not visible from reading the code once — they only surface when you check whether referenced fields/middleware actually exist end-to-end. Rev 5 says explicitly (§16) that nothing should be architected around unverified claims. This closes that loop for the claims Rev 5 already made about the existing codebase, before Phase 0 work starts.

Rev6 — Verification result: Rev 5 holds up

Every other specific codebase claim in Rev 5 — the discarded confidence score in rag.py, the six-branch prompt structure, User.js having no academic fields, the pydantic==1.10.13 pin, the absence of any cron/scheduler or hashing infrastructure, zero automated tests, the closed Notification.type enum, and 100%-hardcoded branding — was independently confirmed against the current repository. No changes to the build order, phases, or design decisions in Rev 5 are needed as a result of this pass.

Rev6 — Correction to §3's audit findings

Rev 5 states: “Jobs and Events are live features, not stubs. They run entirely client-side against external Apify endpoints... Only Scholarships is a genuine placeholder.” Correction: Jobs and Events are not meaningfully more “live” than Scholarships in any way that changes engineering effort. All three have zero backend persistence — there is no Job, Event, or Scholarship Mongo model anywhere in server/models/. Scholarship.jsx (180 lines) is a hardcoded static array; Job.jsx (1,339 lines) and Events.jsx (968 lines) are larger only because they also contain live Apify-fetching and regex-parsing logic that Scholarships never had. The distinction Rev 5 draws (“stub” vs. “live”) is real, but it does not translate into a difference in backend readiness — the rebuild-from-zero cost is closer to equal across all three than §3 implies. Does this change anything in Rev 5? No. §2.3's build-order reasoning (“Scholarships before Jobs”) already rests on Jobs carrying 1,300 lines of legacy code worth consulting during rebuild — which remains true regardless of backend persistence. This correction only affects how §3 should be read, not the plan built on top of it.

Rev6 — New findings (not in Rev 5)

Four items surfaced during this pass that Rev 5's audit did not record. All four sit inside work Rev 5 already scheduled (§13.1's security cluster, §13.1's ChromaDB backup item) — they sharpen what that work needs to do rather than adding new scope.

Rev6 Finding 1 — Login lockout is dead code

Field	Detail
Problem	userController.js implements account lockout (5 failed attempts → 15-minute lock) by reading and writing user.loginAttempts and user.accountLockedUntil. Neither field exists on the User schema. Mongoose silently drops writes to paths not declared in the schema, so the counter never persists across requests and the lock can never actually engage.
Why it wasn't caught	The code reads as correct in isolation — the bug is only visible by cross-referencing the controller against the schema, which single-file review does not surface.
Fix	Either add loginAttempts (Number, default 0) and accountLockedUntil (Date) to User.js, or rewrite the check to count recent failures from the existing LoginEvent collection, which already records every attempt with a success flag and timestamp and needs no schema change.

Field	Detail
Where it fits	Rev 5 §13.1 already lists "Login lockout" as a Now-bucket item, reasoning that the data already exists in LoginEvent. That reasoning is what makes this fixable almost for free — the item changes from "verify it works" to "fix the two dead fields," not a new task.

Rev6 Finding 2 — OTP rate limiter defined, never applied

Field	Detail
Problem	otpLimiter is defined in userController.js and express-rate-limit is installed, but no route attaches it. userRoutes.js has a comment stating intent to apply it to resend-otp and forgot-password — followed by route definitions with no limiter middleware.
Fix	Attach the existing otpLimiter to both routes. No new code to write.
Where it fits	Rev 5 §13.1's "Auth rate limiting" item says to "verify it is actually applied... and wire it up if not." This confirms the second branch is the one that's needed.

Rev6 Finding 3 — CORS confirmed fully open

Field	Detail
Problem	server.js calls app.use(cors()) with no origin allow-list — any origin can call the API. Rev 5 §13.1 listed this as "verify the config is restricted"; it is now confirmed permissive, not merely unverified.
Fix	Restrict to the deployed client origin via a CLIENT_URL env var. Single-tenant, single-frontend (§2.2) — there is no legitimate cross-origin caller.
Where it fits	Pairs naturally with the config-driven branding vars proposed in §11; CLIENT_URL is the same kind of per-deployment setting.

Rev6 Finding 4 — Stray tracked ChromaDB at repo root

Field	Detail
Problem	A second, git-tracked ChromaDB store exists at repo-root chroma_db/chroma.sqlite3 (~168KB), distinct from the live, gitignored working store at python/chroma_db/ (~50MB). It appears to be a stale, accidentally-committed artifact — it is what shows up as "modified" on unrelated commits that touch embeddings.
Fix	Confirm no code path reads from the root-level store, then remove it from git tracking in its own dedicated commit — not folded into an unrelated change, since it's tracked history being deliberately removed, not just a working-tree cleanup.
Where it fits	Do not conflate with §13.1's ChromaDB backup task, which protects the real working store at python/chroma_db/. This finding is about removing cruft, not backing anything up.

This addendum changes no decisions, phases, or scope in Rev 5. It exists so Phase 0 starts from a verified rather than assumed picture of the existing code, per Rev 5 §2.4's own "measure before you change" principle.

PART C — SaaS Pivot: Role-Split & Architecture Plan (original, complete)

Preserved in full from UniAssist_FYP2_SaaS_RoleSplit_Plan.md. Part D corrects a small number of specific technical choices made here (ChromaDB tenancy, tenant-routing default) — see the index above.

UniAssist — SaaS Pivot & Role-Hierarchy Plan

Companion to Rev 5 / Rev 6 — not a replacement. Proposes a new locked decision to replace Rev 5 §2.2.

Project: UniAssist — Centralized AI Student Portal, pivoting from single-tenant to multi-tenant SaaS **Base documents:** UniAssist_FYP2_Spec_Rev5.pdf, UniAssist_FYP2_Spec_Rev6_Addendum.pdf **Date:** September 2026

SaaSPlan §0. What changed and why

Change	Why
Rev 5 §2.2 / §18 ("Single-tenant... not a multi-tenant SaaS... no tenant_id work is needed anywhere") is overturned	Explicit product-direction decision: UniAssist becomes a platform serving many universities, with a new Super Admin role above the existing Administrator role.
A fifth role is added to the four described in Rev 5/6	Super Admin: platform operator. Administrator (Rev 5's "admin"), Department Representative (Rev 5's "staff"), Student, Guest are Rev 5's existing four, now re-scoped to "within one university" instead of implicitly "the one university this deployment serves."

This document does not touch Rev 5's feature content (Scholarships/Jobs/Events, matching engine, chatbot handoff, etc.) or Rev 6's four codebase findings — those stand as written. It answers one question: **given the codebase as it actually is today, what does Super Admin control, what does Administrator control, and what has to change structurally to make that split real rather than cosmetic.**

SaaSPlan §1. Recommendation up front

Tenancy model: database-per-tenant ("Hybrid"). Same app code and one running deployment for everyone, but each university gets its own MongoDB database and its own ChromaDB collection, selected by a tenant-resolution layer at request time. Not full shared-schema multi-tenancy (a universityId column on every collection), not fully separate deployments per university (Rev 5's current model).

Cluster vs. database — one shared cluster, not one cluster per university. "Database-per-tenant" means per-tenant *databases*, not per-tenant *infrastructure*. There is exactly **one** MongoDB cluster (one Atlas cluster, or one self-hosted server) for the whole platform. Inside that one cluster, each university gets its own database — uniassist_maju, uniassist_, etc. — each with its own full set of collections. The connection string only differs in the database-name segment:

```
mongodb://cluster-host.example.com/uniassist_maju
mongodb://cluster-host.example.com/uniassist_<next-university>
```

Same pattern for ChromaDB: one server/process, one collection (or on-disk path) per university. This is what keeps cost and ops work from scaling per tenant — the cluster is billed and maintained once; adding a university adds a database, not a server. Isolation still comes from "different database," which Mongo enforces structurally regardless of the two living on the same cluster — a connection opened against uniassist_maju cannot see uniassist_'s collections even by accident, which is the entire property §1's risk argument above depends on.

Why this over the other two industry-standard options, for this specific project:

- **Shared-schema multi-tenancy** (one DB, tenant_id on every row) is what large-scale SaaS (Slack, Notion) runs at thousands of tenants, because per-tenant infrastructure stops being affordable at that scale. UniAssist will realistically run a handful of pilot universities. At that scale, shared-schema buys nothing but risk: it means adding a tenant filter to every one of the ~30+ endpoints the Explore agent found in adminRoutes.js, issueRoutes.js, departmentRoutes.js etc., in a codebase that Rev 5's own audit confirmed has **zero automated tests**. One missed WHERE universityId = ... is a cross-university student-data leak — the worst possible failure mode for a product selling to institutions that will explicitly ask "is our data isolated from other universities."

- **Separate deployment per tenant** (Rev 5's current, verified-working model) has the opposite problem: it gives no actual control plane. Super Admin would just be an ops dashboard watching independent silos — it could not "create a university" without someone manually redeploying, and there's no single place to see all tenants at once, which is the entire point of the role being requested.
- **Database-per-tenant** is the standard middle ground the industry actually reaches for at this scale, precisely because universities (like healthcare orgs, like enterprise customers generally) are the class of customer that cares about physical data separation, not just a filter promise. Isolation is structural (a different Mongo connection string, a different ChromaDB path) rather than a discipline every future contributor has to maintain correctly forever.

What this means concretely: almost none of Rev 5's per-university schema work changes. User, Department, Issue, Chat, AuditLog, LoginEvent, and the new Job/Scholarship/Event/Source models Rev 5 designs all stay exactly as specified — **no tenant_id field is added to any of them**, because each one lives in a database that only one tenant's data ever touches. The only genuinely new thing is a small **control-plane layer** that decides, per request, which database to talk to — and a Tenant/University registry collection that lives in that control plane, not inside any tenant's own database.

SaaSPlan §2. Current state (verified against the codebase)

An Explore-agent investigation of the live repo confirmed the following, with file:line evidence:

- **Roles today:** exactly three, as a flat string enum on a single User schema — `role: {enum: ["student", "staff", "admin"], default: "student"}` (server/models/User.js:43-48). No separate Staff/Department membership model; "staff" = a User row with `role: "staff"` + a department ObjectId ref.
- **Authorization is flat, not hierarchical:** `requireRole(...allowedRoles)` (server/middlewares/requireRole.js) is a plain allow-list check against that one string. `adminRoutes.js:35` gates its entire router with `requireRole("admin")`. The client mirrors this with a single-role check in `ProtectedRoute.jsx:35`. Neither mechanism has any concept of "a role above admin" — they were built assuming admin is the ceiling.
- **Zero tenant concept anywhere.** A repo-wide grep for `tenant|university_id|universityId|orgId|organizationId|schoolId` returned no matches in server/. Department.js is the only "scoping" model and it is flat and university-wide, not itself scoped to anything.
- **Everything is hardcoded to MAJU, not config-driven, despite Rev 5 §11 planning otherwise:**
 - Student email pattern: `MAJU_EMAIL_REGEX` hardcoded in `server/controllers/userController.js:48`, duplicated client-side in `client/src/utils/validation.js`.
 - Staff email pattern: ``maju.${deptCode.toLowerCase()}.edu`` hardcoded in `adminController.js:22``.
 - University name: hardcoded `"UniAssist" / "MAJU Student Portal"` literals in `client/index.html`, email templates, `notify.js`.
 - Scrape target: `python/config.py`'s `WEBSITES` list hardcodes exactly one entry (`jinnah.edu`), with a comment showing how you'd manually uncomment a second one.
 - One Mongo URI (`server/config/db.js:6`), one ChromaDB path and one collection name `"university_chunks"` (`python/config.py:173-174`), process-wide.
 - **One seed script, one admin, one university:** `seedAdmin.js` reads a single `ADMIN_EMAIL/ADMIN_PASSWORD` from `.env` and ensures exactly one admin exists. There is no concept of seeding multiple universities in one run.

This confirms Rev 6's "verification result: Rev 5 holds up" finding from a different angle — Rev 5 built exactly what it said it would build (a clean single-tenant app), which is precisely why the SaaS pivot is real, additive work, not a small role tweak on top of something already flexible.

SaaSPlan §3. The role hierarchy (hierarchical RBAC)

This is a textbook hierarchical RBAC structure — permissions cascade downward, each role is a strict subset-in-scope of the one above it, and least-privilege is enforced by scoping, not by trust:

Super Admin — the platform. Owns the tenant registry. No single university's

- data is its "home"; it operates one level above all of them.

■

▼

Administrator — one university. Everything Rev 5 already specified as "admin",

■ now scoped to exactly one tenant database. Cannot see, list,

■ or affect any other university.

■

▼

Dept. Representative — one department within one university (Rev 5's "staff").

■ Unchanged from Rev 5 §6.7.

▼

Student — one university, self-scoped. Unchanged from Rev 5 §5/§6/§19.

Guest — no account, scoped to one university's public/admissions

content. Unchanged from Rev 5 §19.

Key architectural point, not just a naming one: Super Admin cannot be "a fourth value in the same role enum" the way student/staff/admin are today. Those three values live *inside* a tenant's own database and are meaningful only relative to that one university. A Super Admin account has to keep existing and working even if a university's database is suspended, migrated, or deleted — so it must live in a genuinely separate place (the control-plane database), not as a row in any tenant's User collection. This also means `requireRole()` and `ProtectedRoute` don't get a fourth string added to their allow-lists — Super Admin gets its own authentication context and its own portal, checked by a different, new middleware that never touches a tenant connection at all. Trying to bolt Super Admin onto the existing flat role check would be the wrong shape for what it actually is.

SaaSPlan §4. Responsibility matrix

Capability	Super Admin	Administrator	Dept. Rep	Student	Guest
Create / suspend / delete a university (tenant)	■	—	—	—	—
Provision a university's first Administrator account	■	—	—	—	—
View list of all universities on the platform	■	—	—	—	—
View/edit another university's data of any kind	■ (only via audited support-access, §7)	—	—	—	—
Configure own university's branding (name, colors, logo, email domain/pattern, support email — Rev 5 §11's 7 fields)	—	■	—	—	—
Manage students (search/filter/block) — Rev 5 "Admin side"	—	■	—	—	—

Capability	Super Admin	Administrator	Dept. Rep	Student	Guest
Create/manage Dept. Rep (staff) accounts, auto-generated credentials	—	■	—	—	—
Departments CRUD	—	■	—	—	—
Content & Source management (Jobs/ Scholarships/Events , §9) for own university	—	■	Event submission only, own dept (§6.7)	—	—
Vector DB / knowledge-base CRUD for own university	—	■	—	—	—
Query analytics, issue analytics, audit logs, login events, chat logs — own university only	—	■	Own department subset	—	—
Manage department inbox, reply to issues, escalation view	—	—	■ (own dept only)	—	—
File issues, chat with assistant, browse/apply Jobs/ Scholarships/Events , profile	—	—	—	■	—
Chat (public-scoped), browse internal scholarships/events/programs (Rev 5 §19)	—	—	—	—	■
Platform-wide health dashboard (all tenants' scrape/LLM/DB status)	■	Own tenant only	—	—	—
Platform-wide audit log (Super Admin's own actions)	■	—	—	—	—
LLM/API quota and provider-key policy across tenants	■	—	—	—	—
Billing / subscription plan per university (if built)	■	View own plan only	—	—	—

The Administrator row is, deliberately, almost identical to Rev 5's existing "Admin side" feature list from §3 — that's the point of the pivot being additive rather than a rewrite: nothing Rev 5 designed for the university-level admin changes in *capability*, only in *scope* (their own tenant DB instead of the only DB that exists).

SaaSPlan §5. What Super Admin actually needs — screens and data

A new, small control-plane database (call it PlatformDB, one collection set, shared, never per-tenant):

- Tenant (**University**) **collection** — slug/subdomain, display name, status (active/suspended/provisioning/offboarded), **the tenant's database name on the one shared cluster** (not a separate host/connection string — see §1's cluster-vs-database note), ChromaDB collection/path, the 7 branding fields Rev 5 §11 already enumerated (now living here instead of .env), plan/tier if billing is ever built, createdAt, provisionedAdministrator ref.
- SuperAdminUser **collection** — deliberately separate from any tenant's User model, per §3's architecture point above.
- PlatformAuditLog **collection** — every Super Admin action (create tenant, suspend tenant, support-access into a tenant, quota change), mirroring the discipline auditAdminWrites already applies at the Administrator level today, but for actions that by definition span or precede any single tenant's own audit log.

Screens: tenant list + health (per-tenant: DB reachable, ChromaDB size, last scrape run, LLM error rate, consecutive-failure sources — this is Rev 5 §9.4's health monitoring, aggregated one level up), tenant creation wizard (replaces the manual seedAdmin.js + hand-edited .env flow with a form: university name → creates Tenant row → creates the tenant's Mongo DB/ChromaDB collection → creates the first Administrator → emails credentials, the same auto-generated-credentials pattern createStaffUser already uses today, one level up), tenant suspend/reactivate, platform audit log viewer, and — if pursued — a support-access action that opens a time-boxed, audit-logged session as that tenant's Administrator (standard SaaS support pattern; needs its own explicit consent/notice story since it is real cross-boundary access).

SaaSPlan §6. Request routing — how one deployment finds the right database

Both Node and Python currently assume exactly one Mongo URI / one ChromaDB path for the process's entire lifetime (server/config/db.js, python/config.py). That has to become a per-request lookup:

1. Incoming request carries tenant context — most naturally a subdomain (maju.uniassist.app) or, for the mobile/PWA client, a tenant claim embedded in the JWT at login.
2. A new Node middleware resolves that context against PlatformDB.Tenant, and attaches a **cached, tenant-specific Mongoose connection** — the same cluster host every time, just a different database name per tenant (mongoose.createConnection(`\${CLUSTER\_URI}/\${tenant.dbName}`), pooled and reused, not reopened per request) — to req. Every existing controller keeps using req-scoped models exactly as it does today — the controllers themselves barely change.
3. Python's FastAPI service needs the equivalent: a dependency that resolves tenant context and picks the right CHROMA_DB_PATH/collection instead of the single hardcoded one in config.py today. The Node↔Python internal-secret bridge (INTERNAL_SECRET, §3 of the codebase report) needs the tenant identifier threaded through it too, since Python currently has no way to know which university's knowledge base a request is even about.
4. Super Admin's own portal talks **only** to PlatformDB — it has no tenant context and, outside the explicit support-access flow in §5, never opens a tenant connection at all.

This is the one genuinely new piece of infrastructure this pivot requires. Everything else in this document is either a scoping change (Administrator's queries now run against their own tenant connection instead of the only connection) or a new, small, additive surface (the Super Admin screens in §5).

SaaSPlan §7. Migration path — sits before Rev 5's Phase 0

Rev 5's own phase plan (§12) assumes a single database throughout. Introducing tenant resolution after Phase 0–7 are built would mean retrofitting every route Rev 5 specifies — far more expensive than building tenant-aware from day one, which is exactly the reasoning Rev 5 itself used for config-driven branding in §11 ("do this now... later it means hunting MAJU through fifty files").

Phase	Work	Why here
T0	PlatformDB + Tenant model + tenant-resolution middleware (Node + Python). Migrate MAJU into "tenant #1" — one row, pointing at the existing Mongo URI and ChromaDB path unchanged. Minimal Super Admin portal: list tenants, view one tenant's health.	Nothing about MAJU's current behavior changes for any existing role — this only proves the plumbing works, before Rev 5's Phase 0 (source verification, Pydantic decision) even starts.
T1	Self-service branding: move Rev 5 §11's 7 fields from .env into Tenant document fields, editable by Administrator. Replace MAJU_EMAIL_REGEX / buildStaffEmail's hardcoded domain with per-tenant fields.	Directly supersedes Rev 5 §11 with the SaaS version of the same idea — same fields, same reasoning, now DB-driven instead of env-driven because there's more than one deployment to configure.
T2	Full tenant-creation wizard (replaces manual seedAdmin.js + .env editing), tenant suspend/reactivate, platform audit log.	This is what makes Super Admin's job actually different from Administrator's — provisioning, not just configuring.
T3 (parallel with or after Rev 5's own phases)	Support-access/impersonation with audit trail; billing/plan schema fields (even without payment integration wired up); cross-tenant health dashboard aggregating Rev 5 §9.4's per-source health one level up.	Lower urgency — nothing downstream blocks on these the way T0/T1 block on everything else.

Once T0 lands, **Rev 5's Phase 0 through Phase 11 proceed completely unchanged** — every model, every pipeline, every phase Rev 5 specifies is written *within* one tenant's database, and T0 is precisely what makes "one tenant's database" a well-defined thing instead of an implicit assumption.

SaaSPlan §8. Open items — decide before building, per Rev 5's own §16 discipline

Nothing here is settled; don't architect around it yet.

- LLM API key strategy across tenants.** Rev 5 already treats free-tier LLM budget as the project's tightest constraint for one university. Multiple tenants sharing one pooled key multiplies that risk; per-tenant keys multiply the "admin-panel-configurable LLM keys" complexity Rev 5 §15 explicitly dropped as not worth it *for one tenant*. This needs a real decision, not a default.
- Self-serve tenant signup vs. manually operated provisioning.** Universities are not a self-serve SaaS sales motion in practice — realistically Super Admin (a person) provisions each new university by hand via the T2 wizard. Confirm this is the assumption before anyone builds payment/plan-enforcement machinery nobody asked for.
- Support-access/impersonation.** Whether Super Admin ever needs to act as a university's Administrator (standard SaaS support pattern) is a real scope decision with real audit/consent implications — don't build the access path before deciding whether it's wanted.
- Realistic tenant count for the FYP demo.** One (MAJU) plus one synthetic second university is very likely enough to *demonstrate* the control plane convincingly (create tenant → its own branding → its own isolated data, live, in front of an examiner — the same kind of strong demo moment Rev 5 §17 already builds around live source-adding). Confirm this before over-building for a tenant count that won't exist during the FYP.

SaaSPlan §9. What does *not* change

- Every feature in Rev 5 §4–§10 (chatbot handoff, failed-question log, SLA escalation, the three new modules, the matching engine, notifications) — unchanged, just running inside one tenant's connection.
- Rev 6's four findings (dead login-lockout fields, unwired OTP limiter, open CORS, stray root ChromaDB) — unchanged, still real, still worth fixing regardless of this pivot. CORS in particular gets *more* important with subdomain-per-tenant routing, not less.

- Dept. Rep / Student / Guest role definitions from Rev 5 §6.7 and §19 — unchanged in capability, only re-scoped to "within one university" instead of "the one university this deployment serves."
- The Administrator's day-to-day feature set — unchanged from Rev 5's existing "Admin side" list; only the underlying connection is now tenant-resolved instead of global.

SaaSPlan §10. Summary of decisions

Area	Decision
Tenancy model	Database-per-tenant ("Hybrid"): one app, one deployment, per-tenant Mongo DB + ChromaDB collection.
Super Admin identity	Separate SuperAdminUser collection in a new PlatformDB, not a 4th value in the existing User.role enum.
Administrator scope	Exactly Rev 5's existing "admin" feature set, scoped to one tenant connection instead of the only connection.
Dept. Rep / Student / Guest	Unchanged from Rev 5 §6.7 / §19, re-scoped to one university.
Request routing	Subdomain or JWT-tenant-claim resolved by new middleware in both Node and Python, attaching a cached per-tenant connection.
Branding	Moves from Rev 5 §11's env vars to per-tenant Tenant document fields, self-service for Administrator.
Build order	New Phase T0–T2 sits <i>before</i> Rev 5's Phase 0; Rev 5's phases proceed unchanged after T0 lands.
Billing / self-serve signup	Not assumed. Manual provisioning by Super Admin is the default until decided otherwise (§8.2).
Open items	LLM key pooling across tenants, support-access/impersonation, realistic demo tenant count — all unresolved, see §8.

PART D — Rev 7: Consolidated Critical Review & Final Decisions (original, complete)

Preserved in full from UniAssist_FYP2_Spec_Rev7_Final.pdf. This is the most current layer — where it disagrees with Parts A/B/C, Part D governs.

UniAssist FYP-2 Spec — Rev 7 (Final) End-to-end consolidation of Rev 5 + Rev 6, the SaaS/Super-Admin pivot, and an independent soundness review Project: UniAssist — Centralized AI Student Portal, multi-tenant SaaS platform Base documents: UniAssist_FYP2_Spec_Rev5.pdf · UniAssist_FYP2_Spec_Rev6_Addendum.pdf · UniAssist_FYP2_SaaS_RoleSplit_Plan.md Date: September 2026

Rev7 §1. Purpose of this document, and how to read it

Three documents exist before this one: Rev 5 (the feature spec — Scholarships/Jobs/Events, the matching engine, chatbot handoff, engineering hardening), Rev 6 (a codebase-verification pass confirming Rev 5's factual claims and recording four new findings), and the SaaS Role-Split Plan (proposing a new Super Admin role above Administrator, for a pivot from single-tenant to multi-tenant). This document does three things none of the previous three did on their own:

- Answers directly: given the codebase as it stands and the plan as written across all three documents, is this the right way to build this — not just "what is the plan" but "does the plan survive scrutiny."
- Merges Rev 5 + Rev 6 + the SaaS plan into one build order and one open-items list, so there is a single document to work from instead of three that each assumed a different subset of the others.
- Adds a critical review section (§5) that did not exist before — specific risks, unverified assumptions, and interactions between the SaaS pivot and Rev 5/6's own decisions that none of the three prior documents caught, because each was written before the next existed. Nothing in Rev 5 or Rev 6 is silently overwritten. Where this document changes a prior decision, it says so explicitly and gives the reason, following the same discipline Rev 5 itself used across its own revision history.

Rev7 §2. Executive verdict

Is the overall approach sound? Yes, directionally — with nine concrete gaps that need to close before build, not after. The core design decisions hold up under scrutiny: source-in-database (Rev 5 §9.2), config-over-code, hash-based change detection, the matching cascade, and the database-per-tenant model for the SaaS layer are all defensible choices that generalize well to each other — source-in-DB in particular becomes automatically per-tenant once the database is per-tenant, at no extra design cost. The role hierarchy (Super Admin → Administrator → Dept. Rep → Student → Guest) is a clean, standard hierarchical RBAC shape with no structural contradiction. What is not yet sound is nine specific places where the SaaS pivot's plan was underspecified against the codebase's actual constraints, found by re-reading Rev 5/6 and the SaaS plan together rather than each in isolation. All nine are fixed in this document (§5) rather than left as vague risk. None of them invalidate the direction — they change specific technical choices within it (e.g. how ChromaDB is tenant-scoped, which tenant-routing mechanism to use, how rate limiting keys work).

The one risk this document cannot resolve on your behalf: total scope against the actual FYP-2 timeline. Rev 5 §14 already states its own feature list is "more work than FYP-1 contained." The SaaS pivot (Rev 7 §5.1) adds a further, non-trivial infrastructure layer on top of that. Section 6 proposes a Must/Nice/Defer cut for the SaaS layer specifically so this is a choice you make deliberately, not a scope creep you discover in week 10.

Rev7 §3. What Rev 5 / Rev 6 got right — reconfirmed, not rehashed

These decisions were checked against the live codebase (by an Explore-agent investigation across this document's preparation) and still hold. They are not re-explained in full here — see Rev 5/6 for the reasoning — only reconfirmed as still valid inputs to this document's build order in §9.

Area	Decision	Still holds because
Sources	Stored in DB, not hardcoded (Rev 5 §9.2)	Confirmed the single best structural decision in the plan — and it is what makes the SaaS pivot's per-tenant sources nearly free: a source row already carries no assumption of being global.

Area	Decision	Still holds because
Change detection	Content-hash per source, skip unchanged (Rev 5 §7.2)	No hashing infrastructure exists yet (Rev 5 audit) — still true, still the right cost-control mechanism given free-tier-only infrastructure.
Matching cascade	Hard rules → embeddings → LLM explanation (Rev 5 §8)	Cheap-first ordering is unaffected by tenancy; each layer just runs against one tenant's data instead of the only data.
Audit logging	auditAdminWrites middleware, auto-logs non-GET admin routes (Rev 5/6)	Confirmed still working exactly as documented — and the same design pattern is what the new PlatformAuditLog (§7) copies for Super Admin actions.
Rev 6 findings	Dead login-lockout fields, unwired OTP limiter, open CORS, stray root ChromaDB	All four independently reconfirmed against the current repo while preparing this document. CORS specifically needs a small revision for the SaaS pivot — see §5.6.
Guest model	Prospective applicant, not a limited student (Rev 5 §19)	Unaffected by tenancy — each tenant's guest funnel is scoped to that tenant's own admissions content, same design, one level down.

Rev7 §4. The SaaS pivot — condensed recap

Rev 5 §2.2/§18 locked in "single-tenant, one university per instance, not a multi-tenant SaaS." That is now overturned by explicit product direction: a new Super Admin role sits above the existing Administrator (Rev 5's "admin"), which is scoped to exactly one university. Department Rep, Student and Guest are unchanged from Rev 5 §6.7/§19, just re-scoped from "the one university this deployment serves" to "the one university this account belongs to."

Rev7 §4.1 Tenancy model

Database-per-tenant on one shared cluster. One MongoDB cluster, one ChromaDB server — each university gets its own Mongo database and its own ChromaDB collection inside that shared infrastructure. Not shared-schema (a tenant_id column on every row, too risky given zero automated tests today), not fully separate deployments per university (Rev 5's current model, which gives no actual control plane for Super Admin to operate).

Rev7 §4.2 Why this over the alternatives

- Shared-schema multi-tenancy needs a correct tenant filter on every one of roughly 30+ endpoints, in a codebase with no test suite — one missed filter is a cross-university data leak.
- Separate deployments per tenant (today's model) gives Super Admin nothing to centrally manage — no single place to see all tenants, no way to provision one without a manual redeploy.
- Database-per-tenant gets structural isolation (a different Mongo connection cannot see another tenant's collections) without the operational cost of separate infrastructure per customer — the standard middle ground for this tenant count and this trust model.

Rev7 §5. Critical review — gaps found by re-reading everything together

These nine items were not visible from any single document — each surfaces only when Rev 5's constraints, Rev 6's verified facts, and the SaaS plan's architecture are checked against each other. Each includes the concrete fix, not just the problem, so this section closes these loops rather than only flagging them.

Rev7 §5.1 — Timeline / scope risk (the one this document cannot resolve for you)

Rev 5 §14 already states its feature list is "more work than FYP-1 contained." The SaaS plan's Phase T0–T2 (tenant-resolution plumbing, self-service branding, tenant-creation wizard) is genuinely new infrastructure sitting before Rev 5's own Phase 0. Combined, total scope is substantially larger than either document proposed alone. Fix: §6 below proposes a Must/Nice/Defer cut for the SaaS layer specifically, mirroring the discipline Rev 5 §14 already applied to its own feature list. Treat §6 as a

decision to make explicitly, not a default to inherit silently.

Rev7 §5.2 — Testing gap is more dangerous for tenant-routing code specifically

Rev 6 confirmed zero automated tests exist anywhere. Rev 5 §13.2 buckets "targeted unit tests" into the Next phase, after core work. Tenant-resolution middleware is different in kind from the rest of the app: a bug in it does not degrade a feature, it leaks tenant A's data into tenant B's session. That is a strictly worse failure class than anything Rev 5/6 catalogued. Fix: add one narrow, mandatory test to Phase T0 itself (not deferred to Rev 5 §13.2's Next bucket): given two seeded tenants, assert that a request authenticated against tenant A's connection can never read or write tenant B's database. This is the single cheapest test in the whole project relative to the severity of what it prevents.

Rev7 §5.3 — Free-tier LLM/API budget was sized for one tenant, not N

Rev 5 §16.6 already flags free LLM tier limits as unverified for one university. The SaaS plan's own open items note multi-tenant multiplies this risk but do not resolve it. Running scraping plus RAG plus chat traffic for two or more universities through the same free-tier keys is a materially harder budget problem than Rev 5 ever sized. Fix: before demoing more than one tenant, re-run Rev 5 §16.6's verification step ("check the providers' own pricing pages") against a two-tenant load estimate, not a one-tenant one. If the free tier cannot cover two tenants' worth of traffic, the FYP demo should show tenant isolation (create tenant, branding, separate data) with the second tenant seeded rather than live-scraped, not two tenants both running live pipelines simultaneously.

Rev7 §5.4 — ChromaDB tenancy mechanism, corrected

The SaaS plan implied a different ChromaDB path per tenant — N separate PersistentClient instances in one Python process — an unverified assumption about concurrent multi-instance safety that Rev 5's own "verify before building" discipline (§16) would have caught. Checked directly against `python/database.py:get_chroma_client()`: ChromaDB already supports multiple named collections inside one persistent store. Correction: use one PersistentClient, one CHROMA_DB_PATH, N collections — one collection per tenant (e.g. `chunks_maju`, `chunks_`) — exactly symmetric with the Mongo "one cluster, one database per tenant" design already agreed. This removes the open question entirely rather than leaving it unverified; no separate ChromaDB path or process-per-tenant is needed.

Rev7 §5.5 — Subdomain-based tenant routing may not be practical on likely FYP hosting

The SaaS plan defaults to subdomain routing (`maju.uniassist.app`). That requires wildcard DNS and a wildcard TLS certificate, not guaranteed on typical free-tier student hosting (Render, Vercel, Railway free plans commonly restrict custom/wildcard subdomains).

Fix: default to JWT-embedded tenant claim as the primary routing mechanism (already the fallback in the SaaS plan) rather than subdomain-as-default. A single shared domain with tenant selected at login, carried in the JWT, needs no DNS or certificate work at all and is strictly simpler to demo. Revisit subdomains only if a specific hosting plan with wildcard support is actually chosen.

Rev7 §5.6 — Rev 6's CORS fix needs to stay tenant-aware

Confirmed directly in `server/server.js:24`: `app.use(cors())` with no origin allow-list, exactly as Rev 6 finding #3 states. Rev 6's proposed fix — restrict to a single `CLIENT_URL` — assumed one frontend origin, true under Rev 5's single-tenant model. Fix: if §5.5's JWT-claim routing is adopted (one shared frontend domain, tenant selected at login), Rev 6's original single-`CLIENT_URL` fix applies completely unchanged — no tenant-aware CORS logic is needed. Flagging this explicitly so it is not mistakenly over-built. It would only need revisiting if genuine subdomain-per-tenant routing were adopted instead.

Rev7 §5.7 — Guest rate limiting keys must include tenant

Confirmed in `server/controllers/userController.js:133` (`otpLimiter`) and Rev 5 §19.6's planned guest chat limiter: `express-rate-limit`'s default key is the requester's IP address alone. The moment a second tenant exists, an IP-only key mixes both tenants' guest traffic into one bucket — a guest hitting tenant A's chatbot from a shared IP (e.g. a campus NAT) would consume quota that should belong only to tenant A. Fix: guest rate limiters (and the OTP limiter, for staff/students) must key on (tenant, IP) or (tenant, session), not IP alone, once §5.5's tenant-claim resolution exists — a small change to the existing `keyGenerator` option, not a redesign.

Rev7 §5.8 — Platform audit logging cannot literally reuse auditAdminWrites

The SaaS plan says the new PlatformAuditLog mirrors the discipline `auditAdminWrites` already applies. Checked directly: `auditAdminWrites` writes into a tenant's own AuditLog collection via that tenant's own Mongo connection. Super Admin's actions happen against PlatformDB, a different connection entirely — the existing middleware instance literally cannot be pointed at

both. Fix: build a second, small middleware (same shape, same redaction logic, different target connection/model) rather than assuming reuse. A few hours of net-new code, correctly scoped into Phase T2 — not a risk, just a correction to language that would otherwise be taken literally.

Rev7 §5.9 — Route namespace separation for Super Admin

Today's adminRoutes.js:35 gates its entire router with requireRole("admin"), mounted at /api/admin. Super Admin's new routes must never be mounted under that same router or path prefix — doing so would either accidentally expose platform actions to a tenant Administrator, or require bolting a second, incompatible auth check onto a router built around one flat role check. Fix: mount Super Admin's routes under a distinct top-level prefix (e.g. /api/platform) with its own dedicated auth middleware that checks a PlatformDB session, never a tenant-scoped req.user.role. State this explicitly in Phase T0 rather than leaving router placement to be decided ad hoc later.

Rev7 §6. Revised scope cut for the SaaS layer — Must / Nice / Defer

Mirrors Rev 5 §14's own discipline, applied to the new SaaS infrastructure specifically, so total project scope is a deliberate choice rather than an accumulation.

Bucket	Items
Must have	PlatformDB + Tenant model. Tenant-resolution middleware (Node + Python) with the isolation test from §5.2. JWT-claim tenant routing (§5.5). ChromaDB one-client/N-collections design (§5.4). Administrator self-service branding (Rev 5 §11's 7 fields, moved to per-tenant DB rows). Route namespace separation (§5.9).
Nice to have	Full tenant-creation wizard UI (a seed script plus manual Tenant-row insert covers the same need for an FYP demo with 1-2 tenants). Platform health dashboard aggregating per-tenant source status. Platform audit log viewer UI (the underlying log can exist without a polished viewer).
Defer past FYP-2	Support-access/impersonation flow. Billing/subscription plan enforcement. Self-serve public tenant signup. Per-tenant LLM key management UI (Rev 5 §15 already dropped this once for one tenant; not more urgent for N tenants without a decided key-pooling strategy, §5.3).

This cut keeps the FYP-demoable core ("here is a platform that provably isolates two universities' data, with self-service branding") while pushing the operationally-heavy, lower-value-for-a-report items (tenant wizard polish, billing) to later — the same logic Rev 5 used to defer cross-encoder re-ranking and engagement feedback loops.

Rev7 §7. Final role & responsibility matrix

Single source of truth, superseding the draft matrix in the SaaS plan. Hierarchical RBAC: each role's scope is a strict subset, in domain, of the one above it.

Capability	Super Admin	Administrator	Dept. Rep	Student	Guest
Create / suspend / delete a university (tenant)	✓	—	—	—	—
Provision a university's first Administrator	✓	—	—	—	—
View list of all universities on the platform	✓	—	—	—	—
View/edit another university's data	audited only	—	—	—	—

Capability	Super Admin	Administrator	Dept. Rep	Student	Guest
Configure own university's branding	—	✓	—	—	—
Manage students (search/filter/block)	—	✓	—	—	—
Create/manage Dept. Rep accounts	—	✓	—	—	—
Departments CRUD	—	✓	—	—	—
Content & source mgmt (Jobs/Scholarships/Events)	—	✓	events only	—	—
Vector DB / knowledge-base CRUD	—	✓	—	—	—
Own-university analytics & audit logs	—	✓	own dept.	—	—
Department inbox, issue replies	—	—	✓	—	—
File issues, chat, browse/apply, profile	—	—	—	✓	—
Public chat, internal scholarships/events	—	—	—	—	✓
Platform-wide health dashboard	✓	own tenant	—	—	—
Platform-wide audit log	✓	—	—	—	—
LLM/API quota policy across tenants	✓	—	—	—	—

Rev7 §8. Final architecture summary

Rev7 §8.1 Cluster vs. database

One shared MongoDB cluster and one shared ChromaDB server for the whole platform — not one cluster per university. Each university gets its own Mongo database on that cluster (e.g. uniassist_maju) and its own ChromaDB collection in that one persistent store (§5.4). Isolation is structural at the database/collection level, not by infrastructure duplication — this is what keeps cost flat as tenants are added.

Rev7 §8.2 Request routing

- Incoming request carries a tenant claim embedded in the JWT at login (§5.5) — no subdomain/DNS dependency.
- A Node middleware resolves that claim against PlatformDB.Tenant and attaches a cached, tenant-specific Mongoose connection to the request — same cluster host, different database name per tenant. Existing controllers keep using req-scoped models unchanged.
- Python's FastAPI service gets the equivalent dependency, resolving tenant context to a ChromaDB collection name instead of the single hardcoded one in python/config.py today. The Node↔Python internal-secret bridge carries the tenant identifier through it.

- Super Admin's portal talks only to PlatformDB, under its own /api/platform prefix (§5.9), and opens a tenant connection only via an explicit, audited support-access action if that feature is ever built (deferred, §6).

Rev7 §8.3 Identity

Super Admin is a separate SuperAdminUser collection inside PlatformDB — not a fourth value in the existing User.role enum (student/staff/admin in server/models/User.js). A Super Admin account must survive a tenant database being suspended or deleted, so it cannot live inside any tenant's own database, and the existing flat requireRole()/ProtectedRoute checks are not extended to a fourth string.

Rev7 §9. Unified build order — Rev 5's phases and the SaaS phases, resolved into one sequence

Phase	Work	Why here
T0	PlatformDB + Tenant model + tenant-resolution middleware (Node & Python) + the tenant-isolation test (§5.2) + JWT-claim routing (§5.5) + ChromaDB one-client/N-collections (§5.4) + /api/platform namespace (§5.9). Migrate MAJU into tenant #1, behavior unchanged for every existing role.	Must land before Rev 5's own Phase 0 — every later phase is written inside one tenant's database, and T0 is what makes that a well-defined thing.
T1	Self-service branding: Rev 5 §11's 7 fields move from .env to Tenant document fields, editable by Administrator. Replace hardcoded MAJU email regex / staff-email domain with per-tenant fields.	Directly supersedes Rev 5 §11 with the SaaS version of the same idea; touches the same files Rev 5 §11 already scheduled.
0 (Rev 5)	Verify every candidate source, resolve Pydantic v1/v2, test Crawl4AI locally.	Unchanged from Rev 5 §12 — now runs per-tenant-aware infrastructure instead of a single global one.
1 (Rev 5)	Unified profile schema + CV parsing + hard-rule matching + golden-set evaluation.	Unchanged from Rev 5.
2 (Rev 5)	Scheduler infrastructure + SLA escalation + issue feedback.	Unchanged from Rev 5.
3 (Rev 5)	Chatbot handoff + failed-question log + revised RAG prompt + follow-up rewriting.	Unchanged from Rev 5.
4-11 (Rev 5)	Source management through PWA, exactly as Rev 5 §12 specifies.	Unchanged from Rev 5 — T0/T1 make no further changes needed to this sequence.
T2	Tenant-creation wizard (Nice-to-have, §6) + tenant suspend/reactivate + platform audit log (its own middleware, §5.8).	Can run in parallel with Rev 5's later phases; nothing downstream blocks on it.
T3	Support-access/impersonation, billing schema fields, cross-tenant health dashboard.	Deferred past FYP-2 per §6 — lowest urgency, nothing blocks on these.

Rev7 §10. Consolidated open items — verify before building

Merges Rev 5 §16's original list with this document's new items. Nothing here is settled.

#	Item	Status
1	Pydantic v1 vs v2 (Rev 5 §16.1)	Unresolved — blocks extraction library choice

#	Item	Status
2	Source viability: robots.txt, ToS, anti-bot, JS-rendering per candidate site (Rev 5 §16.2-5)	Unresolved
3	Free LLM tier limits — now re-scoped to N-tenant load, not one (§5.3)	Unresolved, higher stakes than Rev 5 originally sized
4	MAJU event volume/format (Rev 5 §16.7)	Unresolved
5	Crawl4AI/Playwright resource use on the dev machine (Rev 5 §16.8)	Unresolved
6	ChromaDB multi-collection tenancy	Resolved in this document, §5.4 — one client, N collections
7	Tenant routing mechanism (subdomain vs JWT-claim)	Resolved in this document, §5.5 — JWT-claim by default
8	LLM key pooling strategy across tenants (shared vs per-tenant)	Still unresolved — needs an explicit decision, not a default
9	Support-access/impersonation — wanted at all?	Deferred past FYP-2, §6
10	Realistic tenant count for the FYP demo	Recommended: one seeded, one live (§5.3) — confirm before over-building

Rev7 §11. Master summary of decisions

Area	Decision
Overall verdict	Direction is sound; nine specific gaps closed in this document (§5); scope-vs-timeline is the one open call left to the team (§5.1, §6).
Tenancy model	Database-per-tenant, one shared MongoDB cluster, one shared ChromaDB server.
Super Admin identity	Separate SuperAdminUser collection in a new PlatformDB, not a 4th User.role value.
Administrator scope	Exactly Rev 5's existing admin feature set, scoped to one tenant connection.
Dept. Rep / Student / Guest	Unchanged from Rev 5 §6.7/§19, re-scoped to one university.
Tenant routing	JWT-embedded tenant claim, not subdomains (§5.5) — no DNS/certificate dependency.
ChromaDB tenancy	One PersistentClient, one path, one collection per tenant (§5.4).
CORS	Rev 6's single-CLIENT_URL fix applies unchanged under JWT-claim routing (§5.6).
Rate limiting	Keys must include tenant once a second tenant exists (§5.7).
Platform audit log	Its own middleware and connection — cannot literally reuse auditAdminWrites (§5.8).
Route namespace	Super Admin routes under /api/platform, own auth middleware, never mounted under /api/admin (§5.9).
Build order	T0 and T1 precede Rev 5's Phase 0; Rev 5's phases then proceed unchanged; T2/T3 run in parallel or later.

Area	Decision
Testing	One mandatory tenant-isolation test ships with Phase T0 itself, not deferred (§5.2).
Scope cut	Must/Nice/Defer applied to the SaaS layer specifically (§6) — billing, self-serve signup and impersonation deferred past FYP-2.